NovaBank

A Windows Forms Banking Application

Session: 2025 – 2029

Submitted by:

Umer Nazir 2025-CS-37

Supervised by:

Dr. Maida Shahid

Department of Computer Science

University of Engineering and Technology
Lahore Pakistan

Short Description of Project

NovaBank is a Windows Forms banking application developed in C# using the .NET Framework. It addresses the challenge of manually maintaining banking records by providing a fully digital system in which administrators can manage all accounts and customers can independently handle their own finances. The application follows a clean three-layer architecture comprising the UI, BL, and DL layers, built on the object-oriented programming principles covered in CSC104.

The expected output is a fully functional banking system containing sixteen forms, a login screen with role-based access control, fund transfer capabilities, loan request management, debit card issuance, and bank statement generation for both admins and customers.

Users of Application

1. Admin

The Admin has full control over the system. An admin can view all registered users, remove accounts, reset passwords, monitor every transaction across the system, approve or reject loan requests, manage admin role upgrade requests from customers, view all issued debit cards, and generate bank statements for any account in the system.

2. Customer

The Customer is the standard bank account holder. A customer can log in securely, check their current balance, deposit funds into their account, transfer money to any other registered user, submit a loan application, request a debit card, apply for admin role privileges, and view their personal transaction history filtered by date range.

Functional Requirements

ID	As a	I want to perform	So that I can
1	Admin	Add a new user account	Register new bank customers in the system
2	Admin	Delete a user account	Remove inactive or invalid accounts
3	Admin	Update a user password	Maintain up-to-date login credentials
4	Admin	View all transactions	Monitor all fund movements across the bank
5	Admin	Filter transactions by date and username	Locate specific transaction records quickly
6	Admin	Approve or reject loan requests	Control which loan applications get funded
7	Admin	Approve or reject admin role requests	Manage elevation of customer accounts

8	Admin	View all issued debit cards	Track which users have active cards
9	Admin	Generate a bank statement for any user	Provide official transaction records on demand
10	Customer	Log in securely with credentials	Access my personal banking account
11	Customer	Check my current account balance	Know how much money is available to me
12	Customer	Deposit money into my account	Add funds to increase my available balance
13	Customer	Transfer funds to another account	Send money to other registered users
14	Customer	Apply for a loan	Request financial support from the bank
15	Customer	Apply for a debit card	Obtain a physical card linked to my account
16	Customer	Request admin role privileges	Get elevated access to system features
17	Customer	View my personal transaction history	Review all past deposits and transfers

Wireframes

The following wireframes are actual screenshots of the NovaBank Windows Forms application running on a development machine. Each screen demonstrates the purple-themed sidebar navigation and the light lavender content area that is consistent across all forms.

Figure 1: Login Form

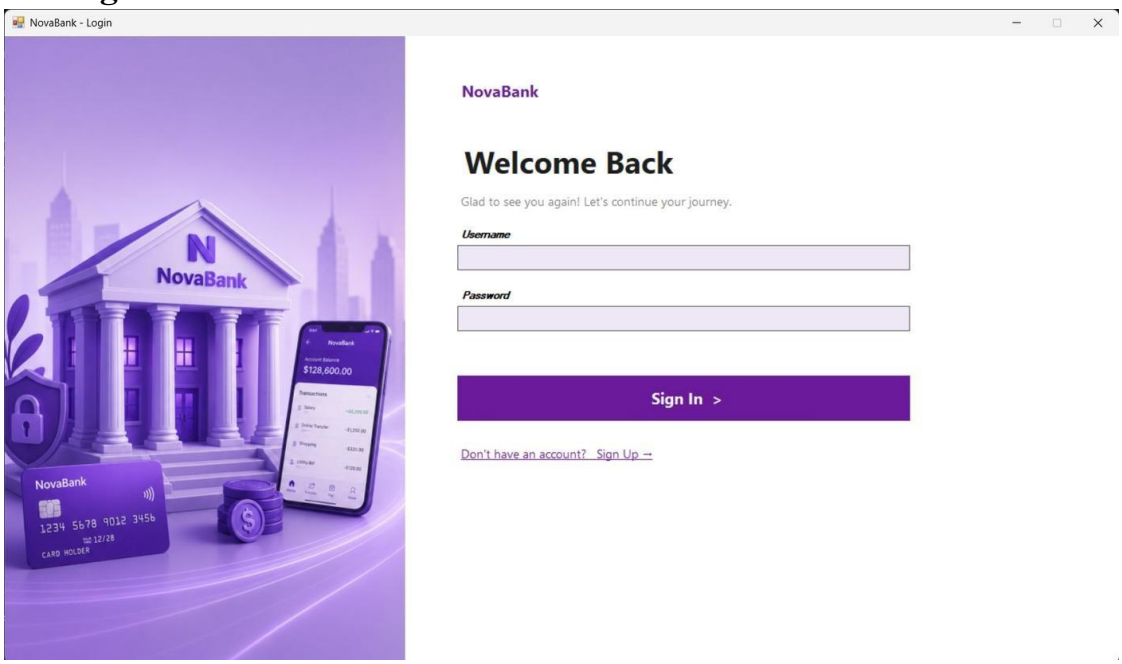


Figure 1: Login Screen — split panel with illustrated bank graphic on the left and the sign-in form on the right

Figure 2: Admin Dashboard

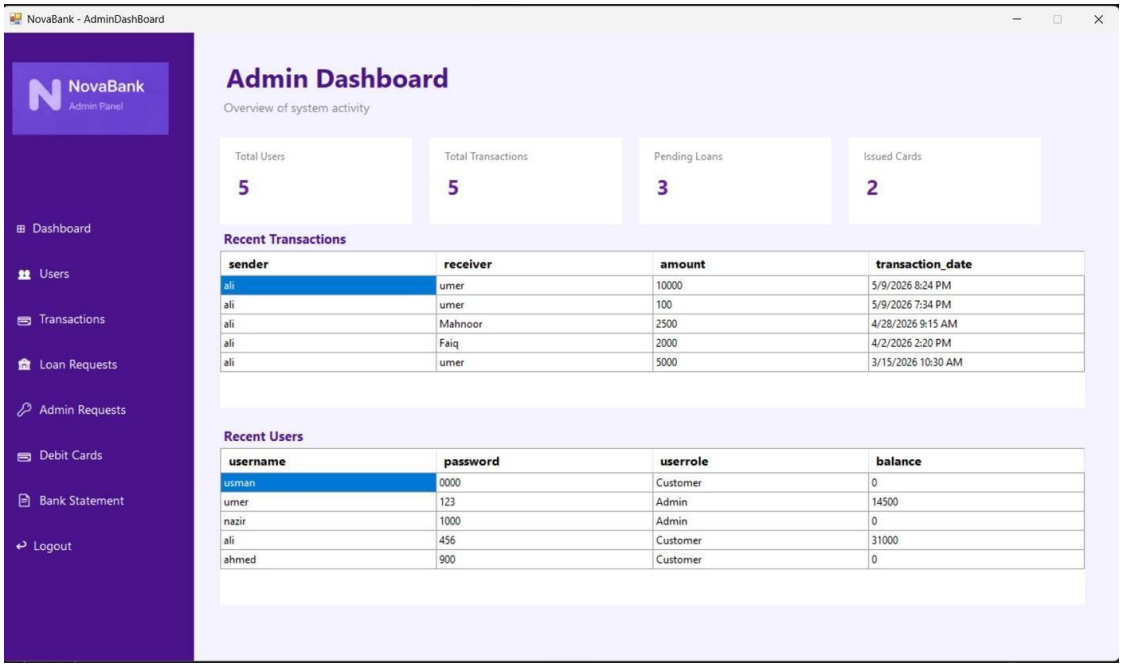


Figure 2: Admin Dashboard — four summary cards showing Total Users, Total Transactions, Pending Loans, and Issued Cards, plus two live data grids

Figure 3: Manage Users (Admin)

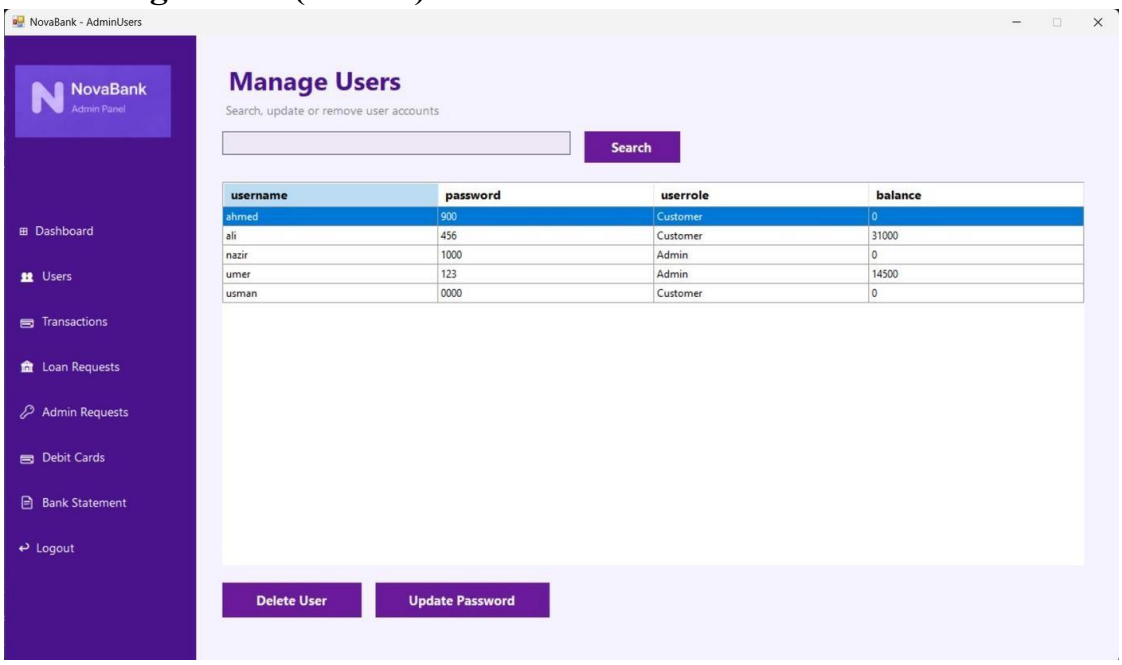


Figure 3: Admin Manage Users — searchable user table with Delete User and Update Password action buttons

Figure 4: All Transactions (Admin)

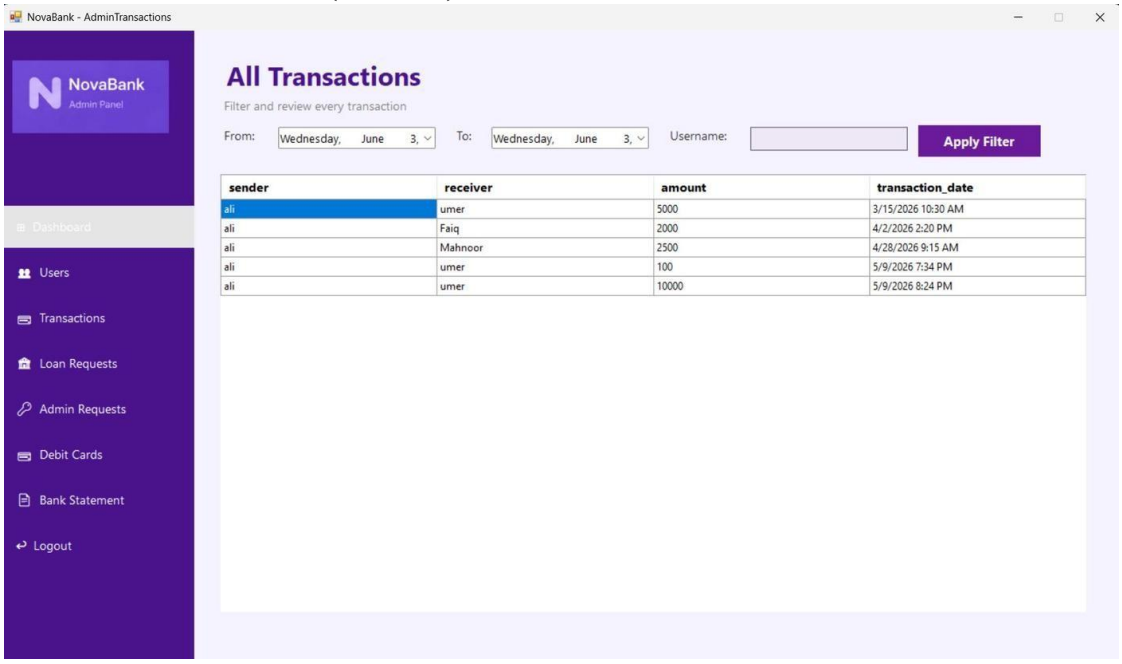


Figure 4: Admin All Transactions — date-range picker and username filter with a sortable transactions grid

Figure 5: Loan Requests (Admin)

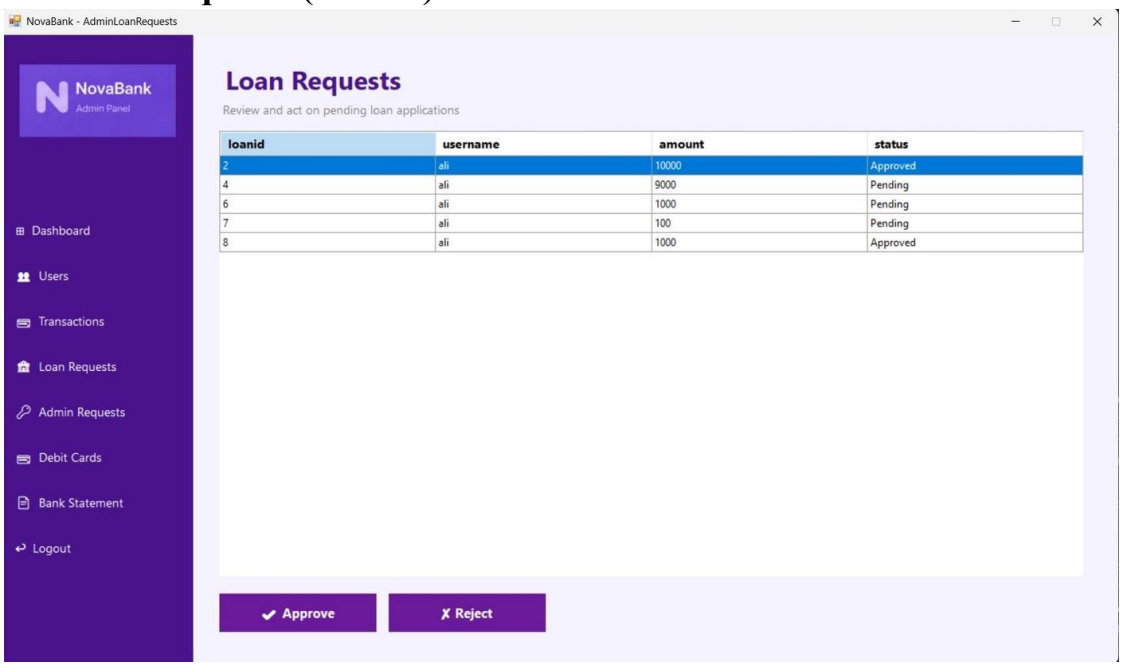


Figure 5: Admin Loan Requests — table showing all loan records with their current status and Approve / Reject action buttons

Figure 6: Admin Role Requests

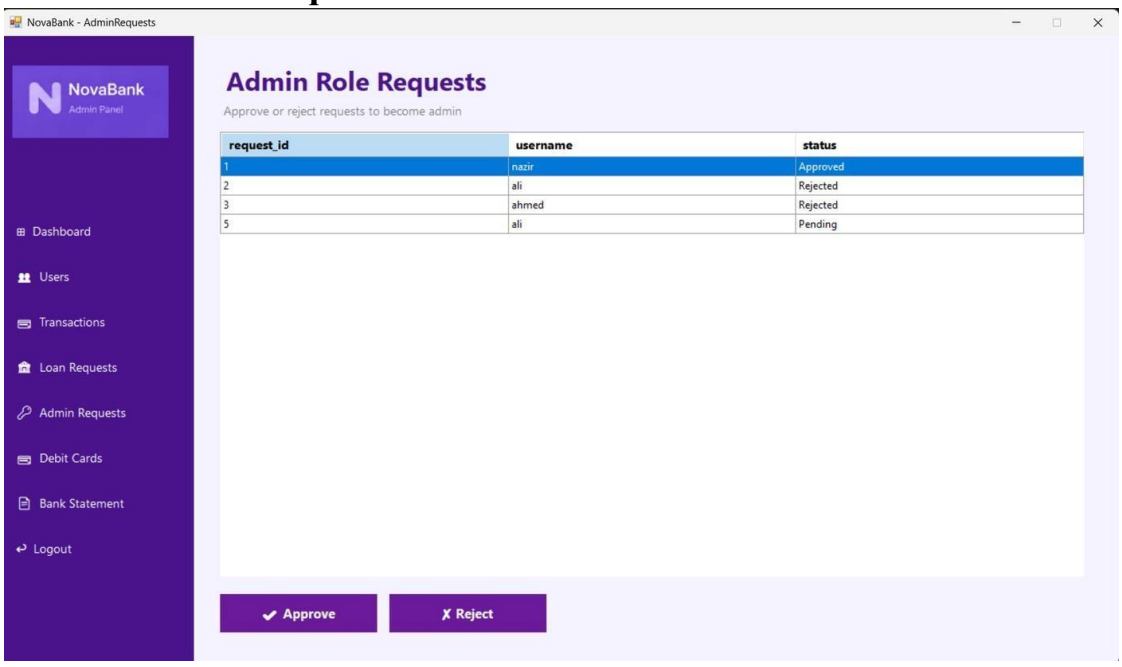


Figure 6: Admin Role Requests — list of customer requests to be promoted to admin, with Approve and Reject controls

Figure 7: Debit Cards (Admin)

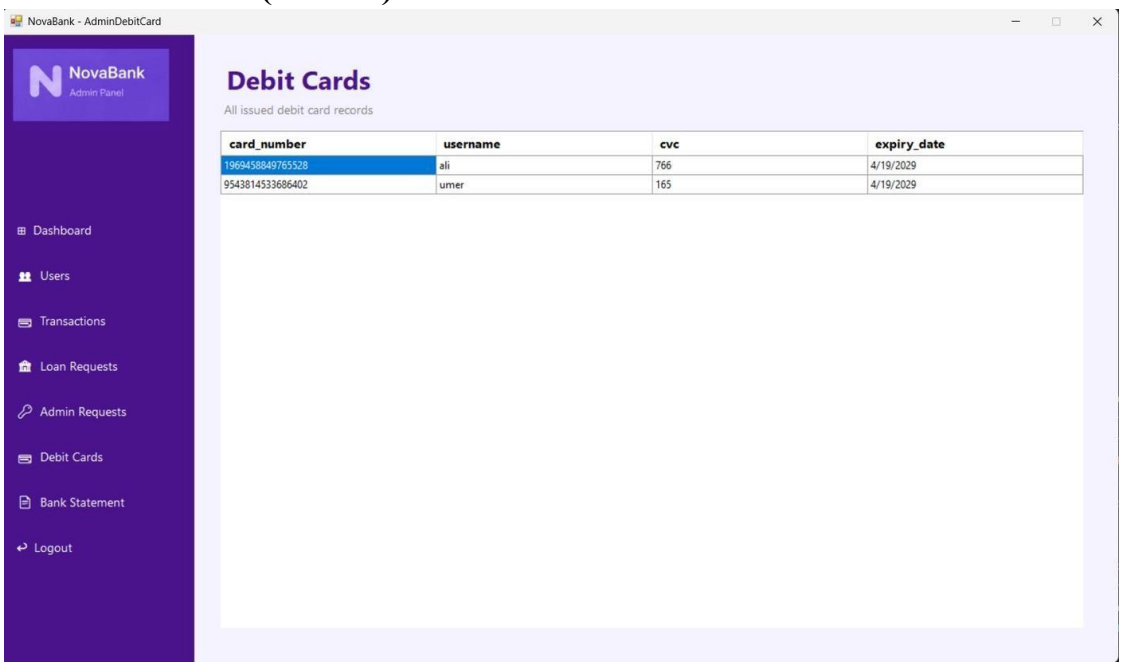


Figure 7: Admin Debit Cards — complete list of all issued debit cards showing card number, holder username, CVC, and expiry

Figure 8: Bank Statement (Admin)

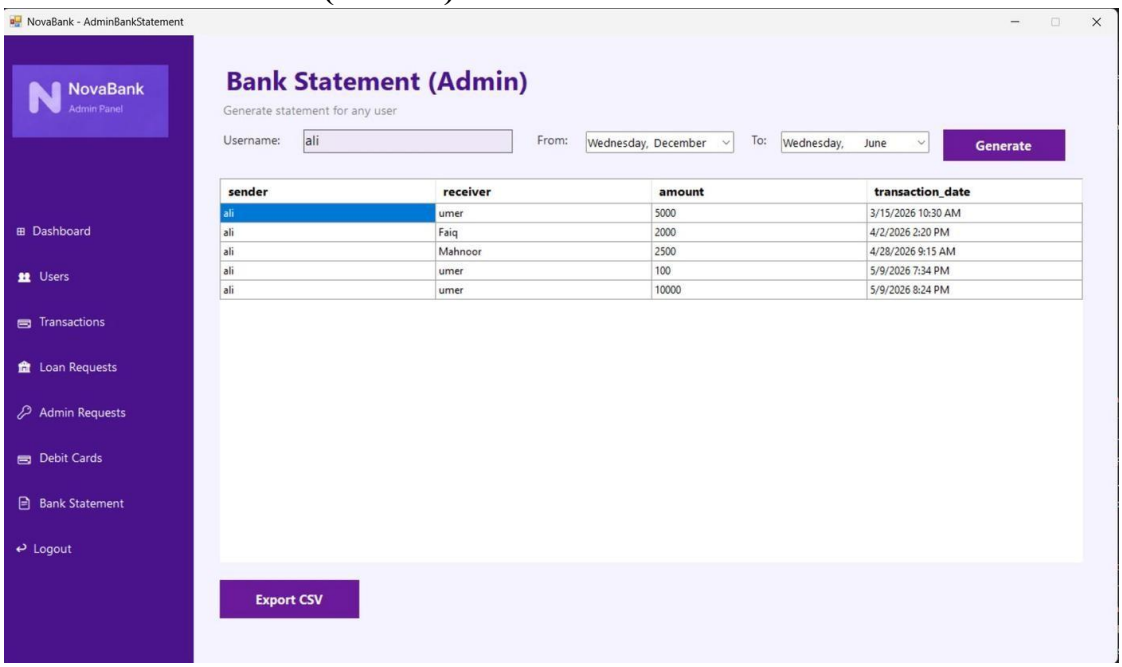


Figure 8: Admin Bank Statement — enter any username and a date range to generate a filtered transaction statement with Export CSV option

Figure 9: Customer Dashboard

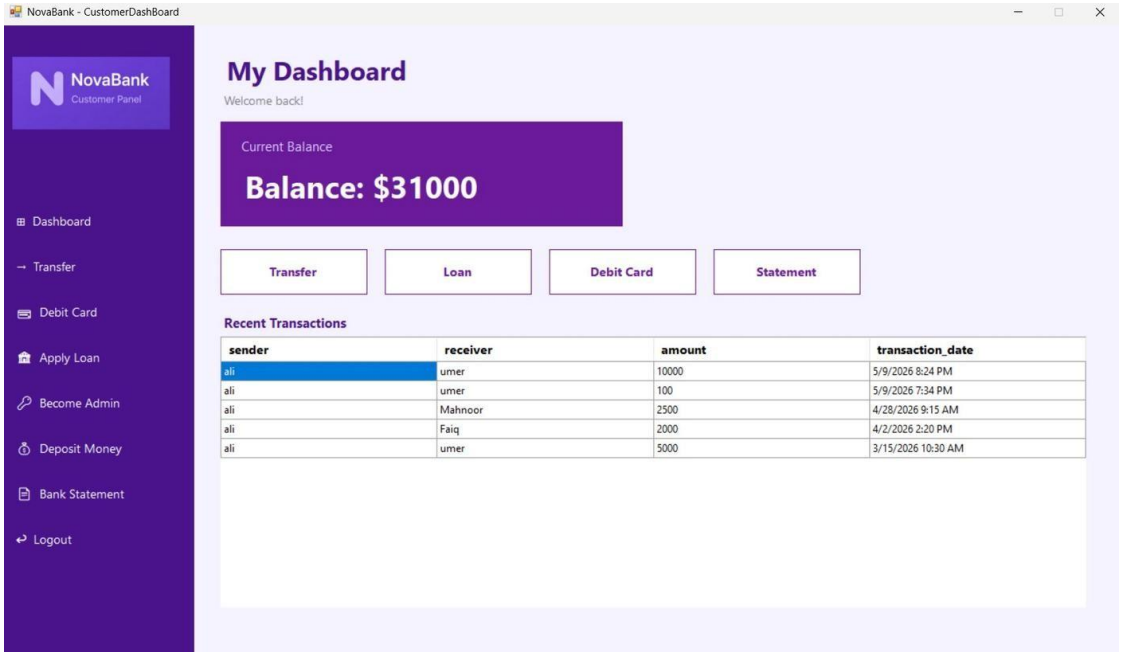


Figure 9: Customer Dashboard — prominently displays current balance, quick-action buttons for Transfer, Loan, Debit Card, and Statement, plus recent transaction history

Figure 10: Transfer Funds (Customer)

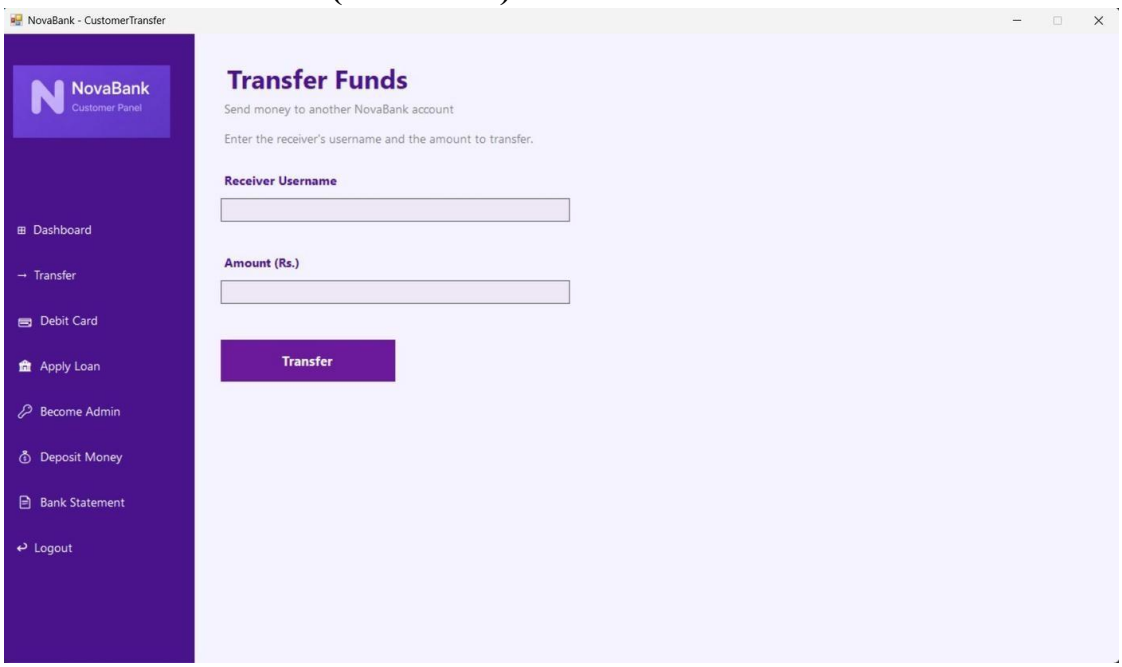


Figure 10: Customer Transfer Funds — input fields for receiver username and amount with a single Transfer action button

Figure 11: Debit Card (Customer)

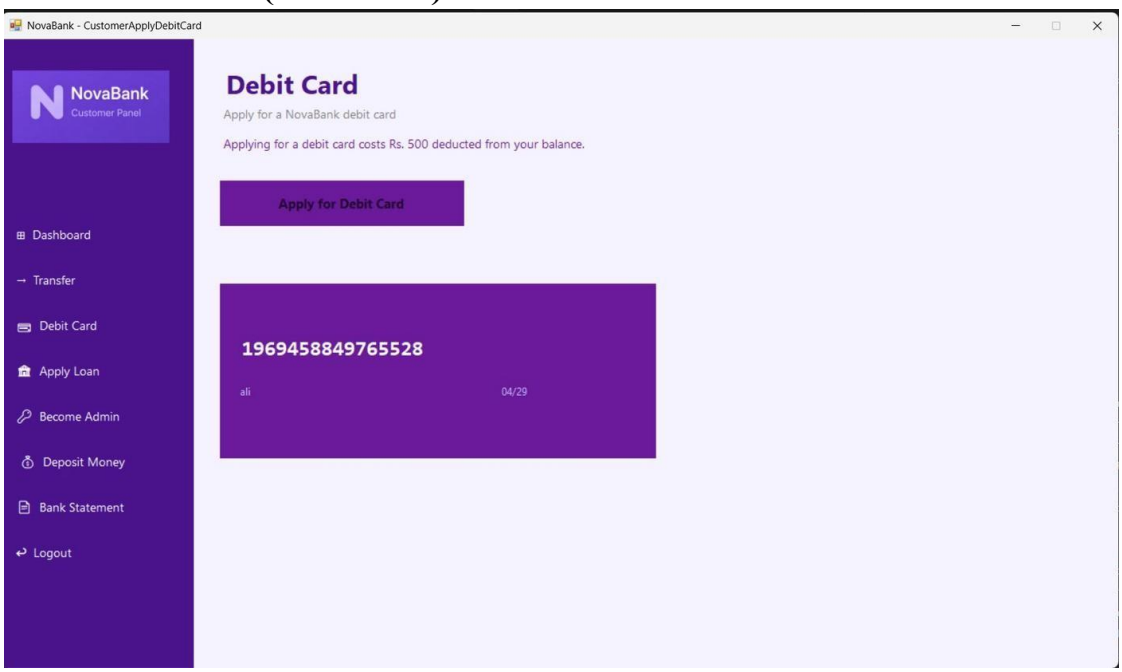


Figure 11: Customer Debit Card — Apply for Debit Card button with a live card mockup displaying the issued card number, holder name, and expiry date

Figure 12: Apply for Loan (Customer)

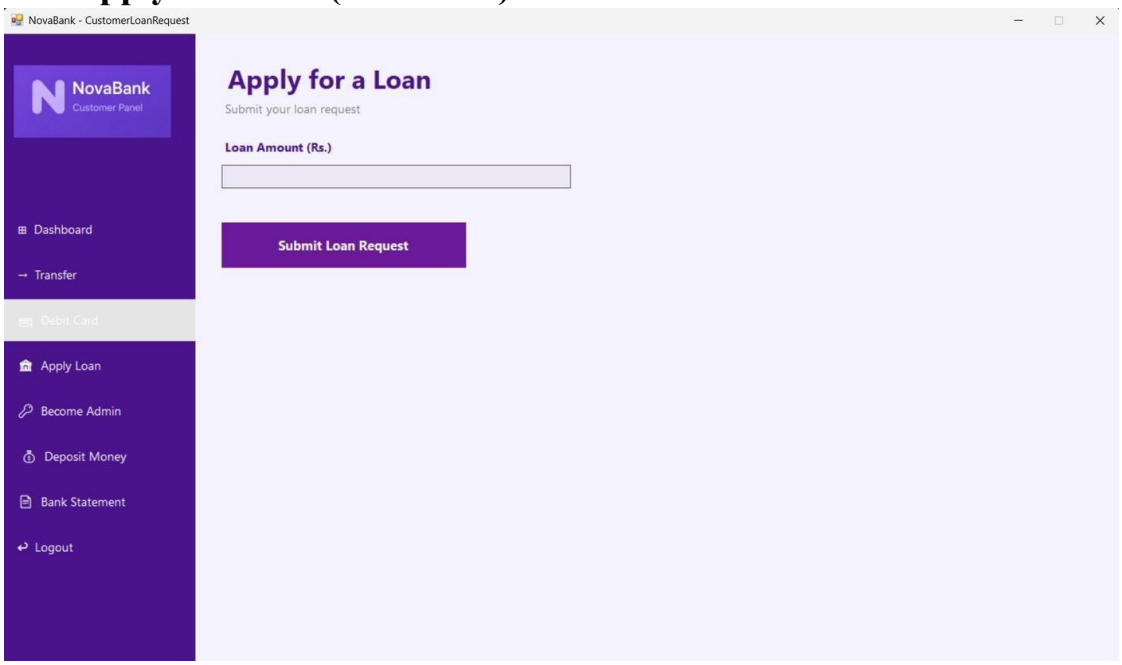


Figure 12: Customer Loan Request — simple loan amount input with a Submit Loan Request button

Figure 13: Become an Admin (Customer)

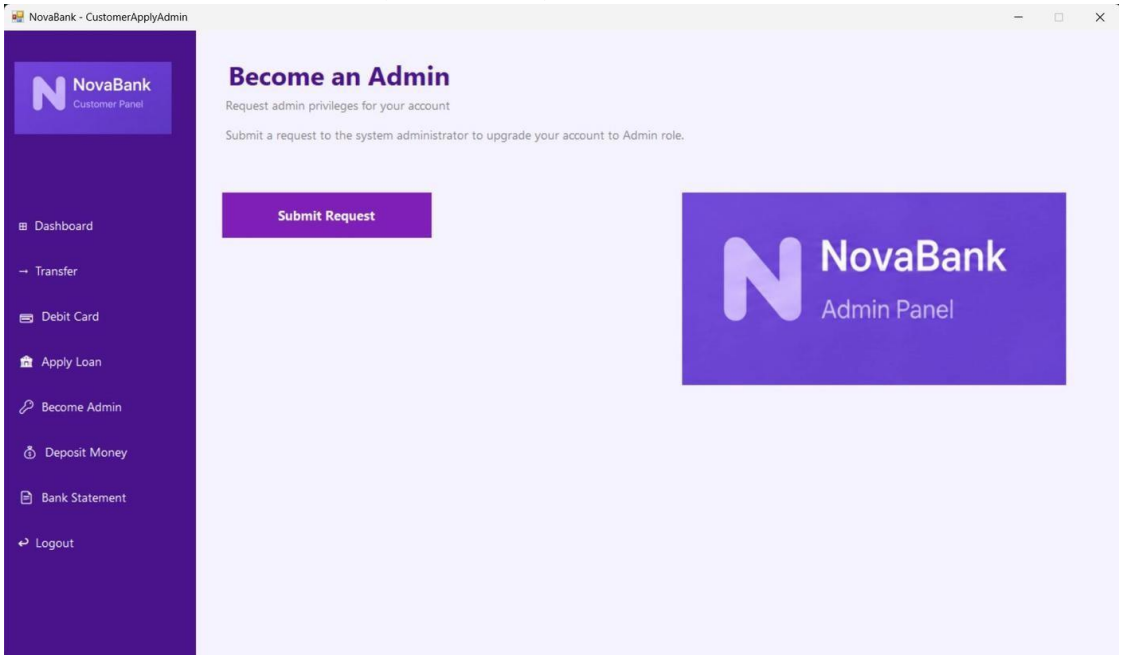


Figure 13: Customer Apply Admin — one-click Submit Request button to apply for admin role elevation

Figure 14: Deposit Money (Customer)

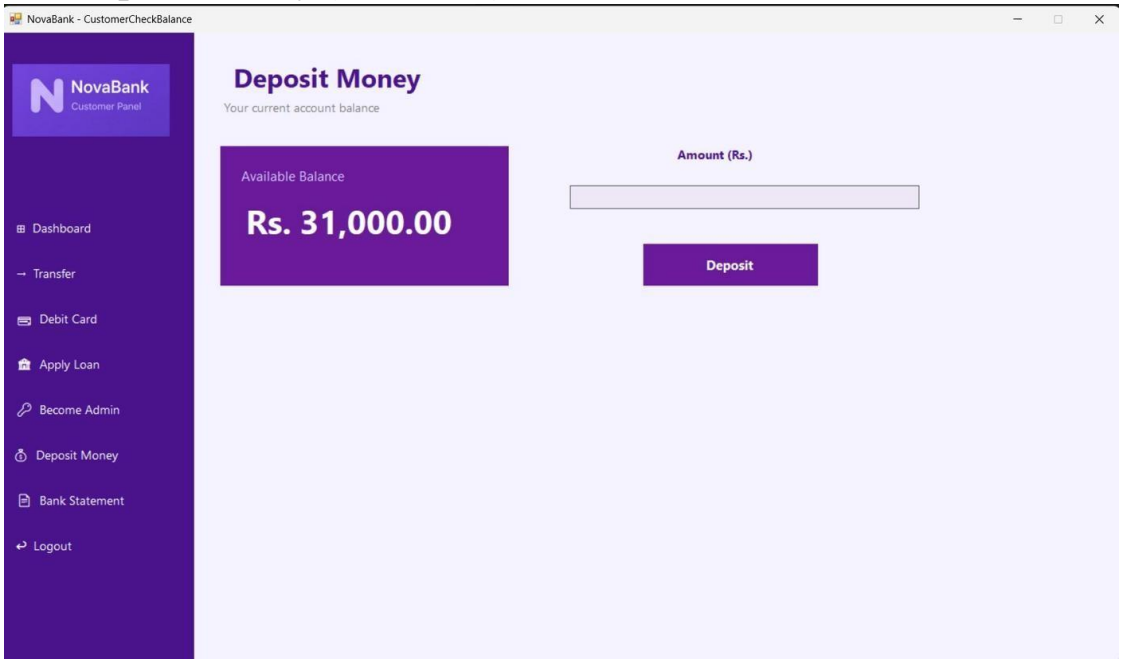


Figure 14: Deposit Money — shows available balance on the left and the deposit amount field with Deposit button on the right

Figure 15: Bank Statement (Customer)

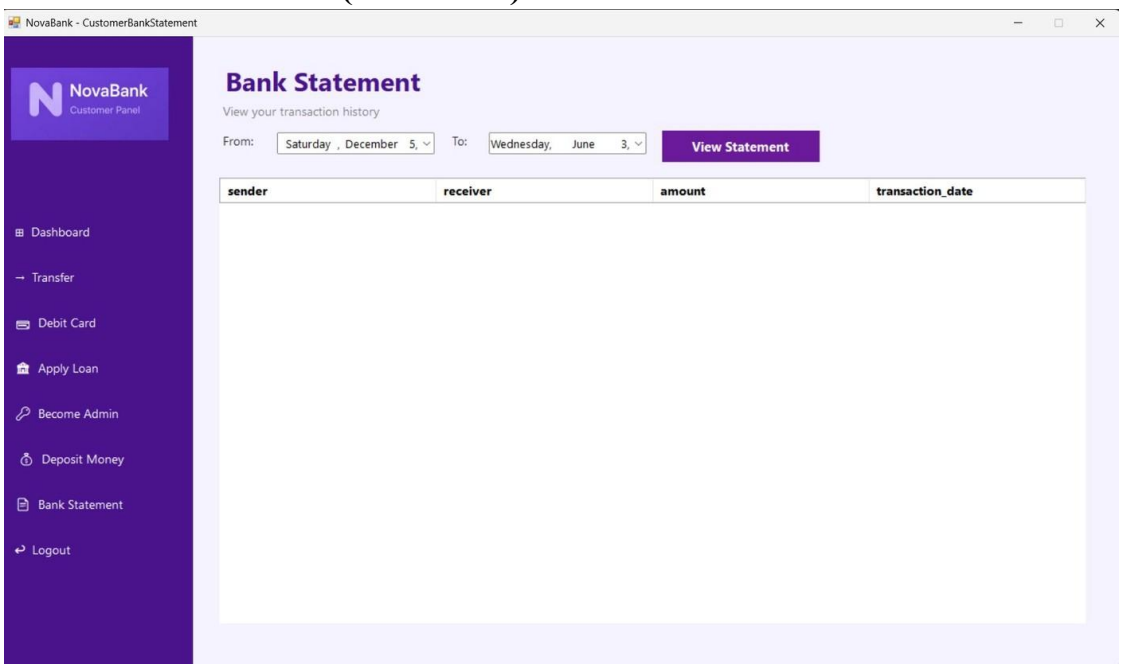
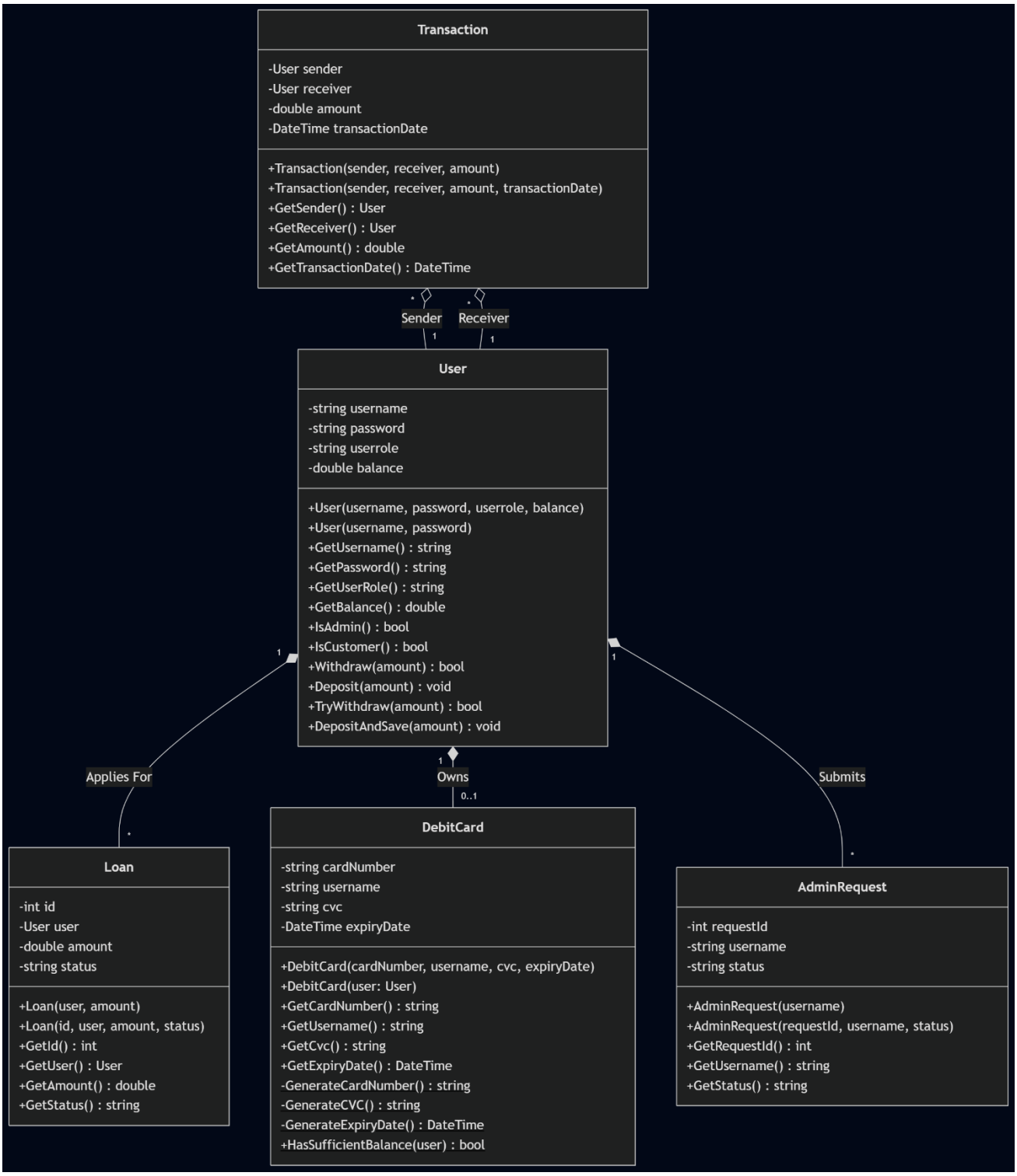


Figure 15: Customer Bank Statement — date-range filter with a View Statement button to retrieve personal transaction history

Class Diagram / Domain Model

The class diagram below illustrates all major classes in NovaBank and the relationships between them. Each class follows the Single Responsibility Principle, meaning every class handles exactly one part of the banking domain.



Organization of Code

The project is divided into three folders inside the Visual Studio Solution Explorer: BL (Business Logic), DL (Data Layer), and UI (Windows Forms). This three-tier separation means that each layer can be read, tested, and modified without touching the others.

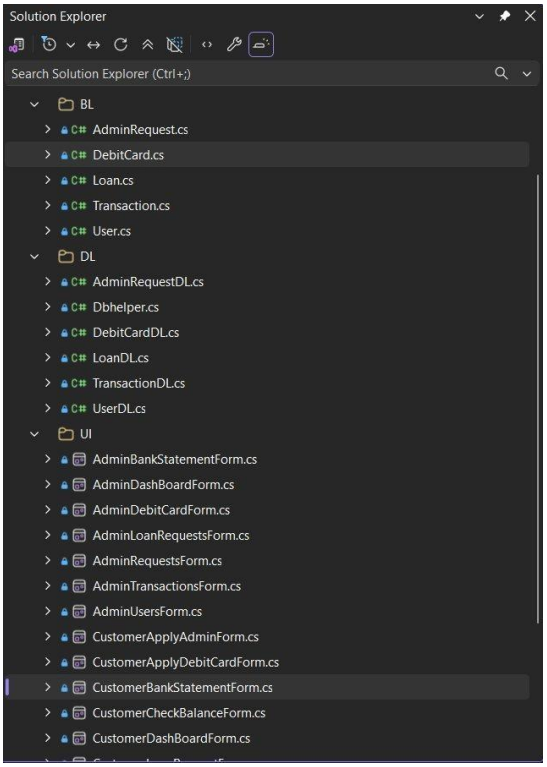


Figure 16: Solution Explorer showing the BL, DL, and UI folder structure with all source files

BL Folder — Contains the five business logic classes: User.cs, Transaction.cs, Loan.cs, DebitCard.cs, and AdminRequest.cs. These classes hold private fields, enforce all domain rules such as balance checks and duplicate card prevention, and expose only what is needed through public getter methods.

DL Folder — Contains DbHelper.cs plus one data-layer class for each BL entity: UserDL.cs, TransactionDL.cs, LoanDL.cs, DebitCardDL.cs, and AdminRequestDL.cs. Every DL class routes its SQL through DbHelper so that the connection string is defined in a single place.

UI Folder — Contains all sixteen Windows Forms files. Each form is responsible only for reading user input and calling the appropriate BL or DL methods. No SQL queries appear in any UI file.

Application Working Flow

The diagram below shows how the application navigates between forms depending on the role of the logged-in user and the actions they choose.

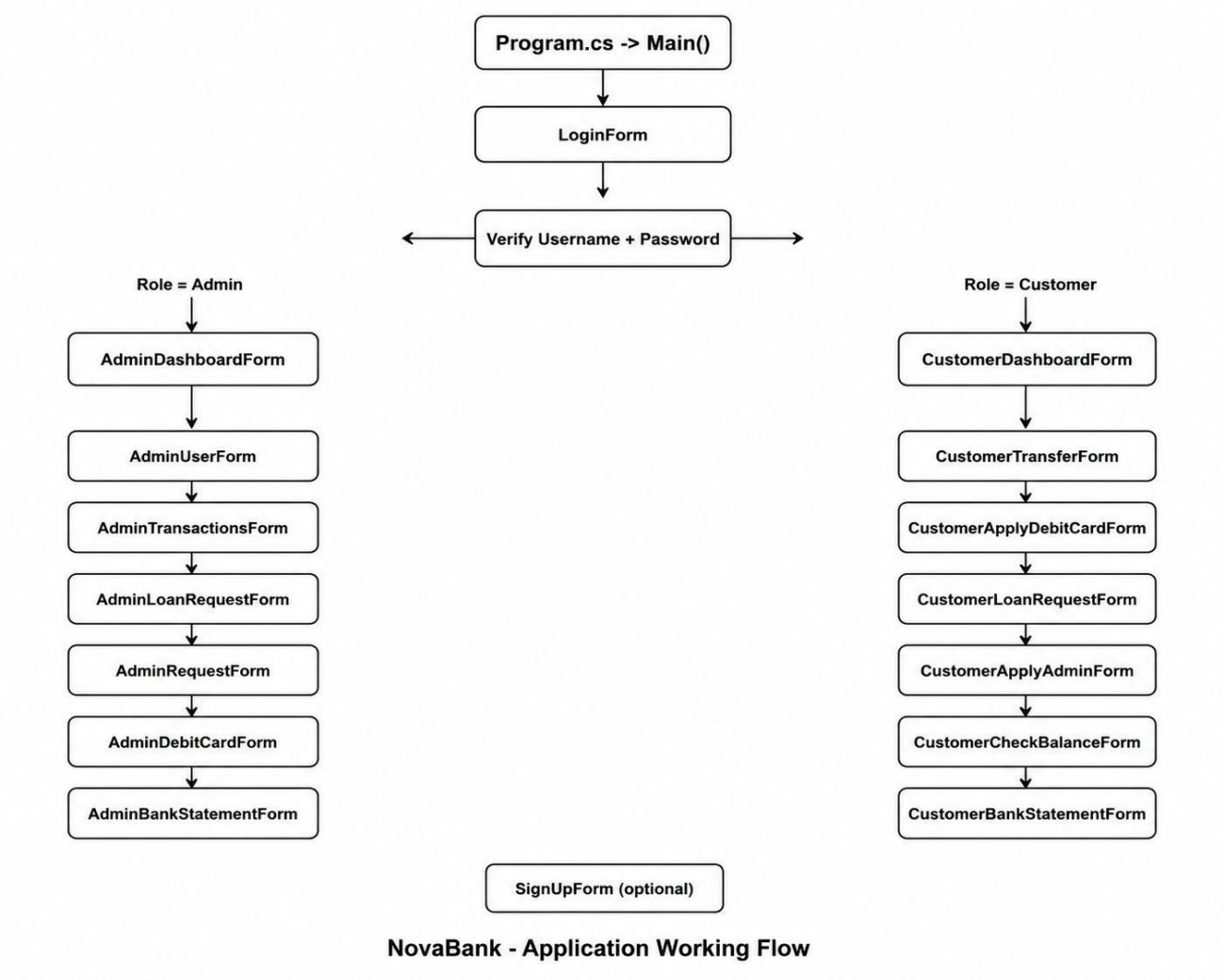


Figure 17: NovaBank Application Working Flow — role-based navigation from Login through Admin and Customer form trees

The entry point is Program.cs, which calls Application.Run() on a new LoginForm. When the user submits their credentials, UserDL.SignIn() queries the database and returns a User object if the credentials are valid. If the returned role is Admin, AdminDashBoardForm is constructed with the logged-in username and displayed. If the role is Customer, CustomerDashBoardForm opens instead. From each dashboard, the sidebar navigation buttons construct and show the corresponding feature form while hiding the dashboard, creating a single-window navigation pattern throughout the application.

Project Architecture

NovaBank uses a three-layer architecture to keep presentation, logic, and data access cleanly separated. Each layer communicates only with the layer directly below it.

UI Layer — Windows Forms

Contains all sixteen WinForms screens. Forms read user input, call BL methods, and display results in labels and DataGridViews. They contain no SQL statements and no business calculations. The purple sidebar with icon labels is replicated consistently across every admin and customer form.

Business Logic Layer (BL)

Contains User, Transaction, Loan, DebitCard, and AdminRequest. Each class enforces its own rules before delegating to the DL. For example, Transaction.Process() calls sender.Withdraw() before calling receiver.Deposit(), ensuring balance integrity without the UI needing to know how.

Data Layer (DL)

DbHelper.cs provides three reusable static methods: ExecuteQuery for SELECT statements that return a DataTable, ExecuteNonQuery for INSERT, UPDATE, and DELETE statements that return row counts, and ExecuteScalar for queries that return a single value. All five domain DL classes call these three methods exclusively, so the connection string and open/close logic are written exactly once.

Database

SQL Server Express is the backend database. The connection string targets a local instance named DESKTOP-MNNND81\SQLEXPRESS with a database called bank. The five core tables are users, transactions, loan, debit_cards, and admin_requests.

GUI Screens and Forms

NovaBank contains a total of sixteen forms split equally between Admin and Customer panels. Every form uses the same dark purple sidebar on the left and a light lavender content area on the right, giving the application a consistent visual identity throughout.

Admin Forms (8 forms)

AdminDashBoardForm displays four live summary cards and two data grids showing recent transactions and recent users. AdminUsersForm offers a search box, a sortable user table, and Delete User and Update Password buttons. AdminTransactionsForm provides a date-range picker and username filter to query any slice of the transaction history. AdminLoanRequestsForm shows all loan records with their current status and Approve or Reject action buttons at the bottom. AdminRequestsForm handles customer requests for admin role elevation using the same approve and reject pattern. AdminDebitCardForm displays all issued card records. AdminBankStatementForm generates filtered statements for any user and includes an Export CSV button.

Customer Forms (8 forms)

CustomerDashBoardForm shows the current balance in a prominent purple card, quick-action shortcut buttons, and a recent transaction grid. CustomerTransferForm accepts a receiver username and an amount to execute a fund transfer. CustomerApplyDebitCardForm shows the card fee notice and renders an animated card mockup after issuance. CustomerLoanRequestForm submits a loan amount for admin review. CustomerApplyAdminForm sends a one-click request for role elevation. CustomerCheckBalanceForm displays the available balance and handles cash deposits. CustomerBankStatementForm allows date-filtered self-service statement retrieval.

Event Driven Programming

Button Click Events

Every user action is handled through a Click event on a Button control. In LoginForm, btnSignIn_Click constructs a User object from the username and password textboxes, passes it to UserDL.SignIn(), and routes to the correct dashboard based on the returned role. In CustomerTransferForm, btnTransfer_Click retrieves both the sender and receiver User objects, calls Withdraw() on the sender and Deposit() on the receiver, then calls UserDL.UpdateBalance() for both accounts and TransactionDL.AddTransaction() to persist the record.

Form Load Events

Each form's constructor serves as its load handler. AdminDashBoardForm's constructor calls LoadStats(), which queries the counts for users, transactions, pending loans, and issued cards and populates the four summary labels. It then sets the DataSource of both DataGridViews from TransactionDL.GetTop5UserTransactions() and UserDL.GetTop5RecentUsersDataTable() so the dashboard is fully populated the moment it opens.

Validation Events

Input is validated before any action is executed. CustomerTransferForm uses double.TryParse() on the amount field and shows a warning MessageBox if parsing fails, preventing the transfer from proceeding. SignUpForm compares the password and confirm-password fields and blocks registration if they do not match. AdminLoanRequestsForm checks the current status of a selected loan before allowing approve or reject, and displays an informative message if the loan is already in a final state.

Database Connectivity

NovaBank connects to SQL Server using the Microsoft.Data.SqlClient package. All database access is channelled through a single static DbHelper class so that the connection string is defined once and every DL class benefits from the same open-close-dispose pattern automatically.

Connection String

```
private static string connString =
    "Server=DESKTOP-MNNND81\\SQLEXPRESS;Database=bank;
     Trusted_Connection=True;TrustServerCertificate=True;";
```

DbHelper Methods

ExecuteQuery(string query) opens a connection, executes a SELECT statement using SqlDataAdapter, fills a DataTable, and returns it. ExecuteNonQuery(string query) opens a connection, executes an INSERT, UPDATE, or DELETE statement, and returns the number of affected rows. ExecuteScalar(string query) returns a single object value suitable for COUNT or SUM queries. All three methods use a using block so the SqlConnection is always disposed even if an exception occurs.

Database Tables

Table Name	Description
users	Stores username, password, userrole (Admin or Customer), and balance for every registered account
transactions	Records every fund transfer with sender, receiver, amount, and transaction_date columns
loan	Stores loan applications with loanid, username, requested amount, and approval status
debit_cards	Holds issued card details including card_number, username, cvc, and expiry_date
admin_requests	Tracks customer requests to become admin with request_id, username, and current status

Object Oriented Programming Concepts

Classes and Objects

Every real-world banking entity is represented as a class. The five main BL classes are User, Transaction, Loan, DebitCard, and AdminRequest. When the application needs to work with a specific customer, it creates a User object by calling UserDL.GetUser(username), which returns a fully populated User instance built from the corresponding database row.

Encapsulation

All fields across every BL class are declared private. External code can only read values through dedicated public getter methods such as GetUsername(), GetBalance(), GetAmount(), and GetStatus(). Modification is possible only through controlled methods. In User.cs, for example, the balance field can only change via Withdraw() or Deposit(), both of which enforce domain rules: Withdraw() returns false and does nothing if the requested amount exceeds the current balance, protecting account integrity at the object level.

Abstraction

Forms have no awareness of SQL. A form like CustomerTransferForm calls Transaction.Transfer(), which internally calls UserDL.GetUser() and TransactionDL.AddTransaction(). The form simply checks the boolean result and shows a success or failure message. Each layer hides the complexity of the layer below it, which is abstraction applied at the architectural level.

Constructors and Overloading

Most BL classes provide two constructors to handle different creation scenarios. The User class has one constructor for registering a brand-new account (username and password only, role defaults to Customer and balance to 0.0) and a second for loading an existing account from the database (all four fields required). DebitCard has a constructor that accepts four explicit parameters for loading saved cards and a second constructor that accepts only a User object, generating the card number, CVC, and expiry date automatically and deducting the Rs. 500 fee in the same step.

Exception Handling and Validations

Input Validation

Every form that collects numeric input validates it before acting. double.TryParse() is applied to all amount fields so that a non-numeric entry can never cause a runtime crash. Null checks are performed on every object returned from the DL layer. If UserDL.GetUser() returns null, the calling form shows an informative MessageBox error and cancels the operation rather than attempting to call methods on a null reference.

Exception Handling

All SqlConnection objects in DbHelper are wrapped in using blocks. This guarantees that connections are closed and resources are released even when a query throws an exception. The try-catch pattern applied around database operations ensures that if the SQL Server instance is unavailable or a query violates a constraint, the application displays a user-friendly error message instead of terminating with an unhandled exception dialog.

Null Safety

Every method that retrieves data from the database performs a null or row-count check before constructing a return value. User.VerifyUser() calls UserDL.GetUser() and immediately returns null if no matching row is found, without attempting to compare the password. The calling form then checks the return value before proceeding. This single pattern, applied consistently across all five DL classes, eliminates NullReferenceException as a source of crashes throughout the application.

Complete Code of the Windows Forms Application

The complete source code of NovaBank is listed below. Auto-generated Designer.cs files are excluded as they contain only Visual Studio layout markup. All business logic, data access, and UI event handler code is included in full.

Program.cs

```
using System;
using System.Windows.Forms;

namespace BankUI
{
   internal static class Program
   {
      [STAThread]
      static void Main()
      {
         Application.EnableVisualStyles();
         Application.SetCompatibleTextRenderingDefault(false);
         Application.Run(new LoginForm());
      }
   }
}
```

BL / User.cs

```
using System;
using BankUI.DL;

namespace BankUI.BL
{
   public class User
   {
      private string username;
      private string password;
      private string userrole;
      private double balance;

      public User(string username, ststringassword, stristringrrole, double balance)
      {
         this.username = username;
         this.password = password;
         this.userrole = userrole;
         this.balance = balance;
      }

      public User(string username, ststringassword)
```

```
{
    this.username = username;
    this.password = password;
    this.userrole = "Customer";
    this.balance = 0.0;
}

public string GetUsername()
{
    return this.username;
}
public string GetPassword()
{
    return this.password;
}
public string GetUserRole()
{
    return this.userrole;
}
public double GetBalance()
{
    return this.balance;
}
public bool IsAdmin()
{
    if (userrole == "Admin")
    {
        return true;
    }
    return false;
}
public bool IsCustomer()
{
    if (userrole == "Customer")
    {
        return true;
    }
    return false;
}
public bool Withdraw(double amount)
{
    if (amount <= balance)
    {
        balance -= amount;
        return true;
    }
    return false;
}
```

```
        public void Deposit(double amount)
        {
            balance += amount;
        }
        public double checkBalance()
        {
            return balance;
        }


    }
}
```

BL / Transaction.cs

```
using BankUI.DL;
using System;
using System.Collections.Generic;
using System.Text;

namespace BankUI.BL
{
    public class Transaction
    {
        private User sender;
        private User receiver;
        private double amount;
        private DateTime transactionDate;

        public Transaction(User sender, User receiver, double amount)
        {
            this.sender = sender;
            this.receiver = receiver;
            this.amount = amount;
            this.transactionDate = DateTime.Now;
        }

        public Transaction(User sender, User receiver, double amount, DateTime transactionDate)
        {
            this.sender = sender;
            this.receiver = receiver;
            this.amount = amount;
            this.transactionDate = transactionDate;
        }

        public User GetSender()
        {
            return sender;
```

```
        }

        public User GetReceiver()
        {
            return receiver;
        }

        public double GetAmount()
        {
            return amount;
        }

        public DateTime GetTransactionDate()
        {
            return transactionDate;
        }

        public bool Process()
        {

            if (amount > 0 && sender.Withdraw(amount))
            {
                receiver.Deposit(amount);
                return true;
            }
            return false;
        }


    }
}
```

BL / Loan.cs

```
using BankUI.DL;
using System;
using System.Collections.Generic;
using System.Text;

namespace BankUI.BL
{
    public class Loan
    {
        private int id;
        private User user;
        private double amount;
        private string status;
```

```
        public Loan(User user, double amount)
        {
            this.user = user;
            this.amount = amount;
            this.status = "Pending";
        }

        public Loan(int id, User user, double amount, string status)
        {
            this.id = id;
            this.user = user;
            this.amount = amount;
            this.status = status;
        }

        public int GetId()
        {
            return id;
        }

        public User GetUser()
        {
            return user;
        }

        public double GetAmount()
        {
            return amount;
        }

        public string GetStatus()
        {
            return status;
        }

    }
}
```

BL / DebitCard.cs

```
using BankUI.DL;
using System;

namespace BankUI.BL
{
    public class DebitCard
    {
        private string cardNumber;
        private string username;
```

```
private string cvc;
private DateTime expiryDate;

public DebitCard(string cardNumber, ststringsername, stristring, DateTime expiryDate)
{
   this.cardNumber = cardNumber;
   this.username = username;
   this.cvc = cvc;
   this.expiryDate = expiryDate;
}

public DebitCard(User user)
{
   this.username = user.GetUsername();
   this.cardNumber = GenerateCardNumber();
   this.cvc = GenerateCVC();
   this.expiryDate = GenerateExpiryDate();

   user.Withdraw(500);
   UserDl.UpdateBalance(user.GetUsername(), user.GetBalance());
   DebitCardDl.AddDebitCard(this);
}
public string GetCardNumber()
{
   return this.cardNumber;
}
public string GetUsername()
{
   return this.username;
}
public string GetCvc()
{
   return this.cvc;
}
public DateTime GetExpiryDate()
{
   return this.expiryDate;
}

private static string GenerateCardNumber()
{
   Random rand = new Random();
   string number = "";
   for (int i = 0; i < 16; i++)
      number += rand.Next(0, 10).ToString();
   return number;
}
```

```
        private static string GenerateCVC()
        {
            return new Random().Next(100, 999).ToString();
        }

        private static DateTime GenerateExpiryDate()
        {
            return DateTime.Now.AddYears(3);
        }

        public static bool HasSufficientBalance(User user)
        {
            return user.GetBalance() >= 500;
        }

    }
}
```

BL / AdminRequest.cs

```
using BankUI.DL;
using System;
using System.Collections.Generic;

namespace BankUI.BL
{
    public class AdminRequest
    {
        private int requestId;
        private string username;
        private string status;

        public AdminRequest(string username)
        {
            this.username = username;
            this.status = "Pending";
        }

        public AdminRequest(int requestId, string username, ststringtatus)
        {
            this.requestId = requestId;
            this.username = username;
            this.status = status;
        }

        public int GetRequestId()
        {
            return requestId;
        }
```

```
        public string GetUsername()
        {
            return username;
        }
        public string GetStatus()
        {
            return status;
        }

    }
}
```

DL / DbHelper.cs

```
using Microsoft.Data.SqlClient;
using System.Data;


namespace BankUI.DL
{
    public static class DbHelper
    {
        private static string connString = "Server= DESKTOP-
MNNND81\\SQLEXPRESS;Database=bank;Trusted_Connection=True;TrustServerCertificate=True
;";

        public static SqlConnection GetConnection()
        {
            return new SqlConnection(connString);
        }

        public static DataTable ExecuteQuery(string query)
        {
            using (SqlConnection conn = GetConnection())
            {
                conn.Open();
                SqlCommand cmd = new SqlCommand(query, conn);
                SqlDataAdapter adapter = new SqlDataAdapter(cmd);
                DataTable dt = new DataTable();
                adapter.Fill(dt);
                return dt;
            }
        }

        public static int ExecuteNonQuery(string query)
        {
            using (SqlConnection conn = GetConnection())
            {
```

```
            SqlCommand cmd = new SqlCommand(query, conn);
            conn.Open();
            int rowsAffected = cmd.ExecuteNonQuery();
            conn.Close();
            return rowsAffected;
        }
    }

    public static object ExecuteScalar(string query)
    {
        using (SqlConnection conn = GetConnection())
        {
            SqlCommand cmd = new SqlCommand(query, conn);
            conn.Open();
            object result = cmd.ExecuteScalar();
            conn.Close();
            return result;
        }
    }
  }
}
```

DL / UserDL.cs

```
using BankUI.BL;
using System.Collections.Generic;
using System.Data;

namespace BankUI.DL
{
  public class UserDl
  {
    private static List<User> usersList = null;

    public static List<User> UsersList
    {
      get
      {
        if (usersList == null) usersList = GetAllUsers();
        return usersList;
      }
    }

    public static void RefreshUsers()
    {
      usersList = null;
    }
```

```
        public static User SignIn(User userCredentials)
        {
            string query = $"SELECT * FROM users WHERE username =
'{userCredentials.GetUsername()}' AND password = '{userCredentials.GetPassword()}'";
            DataTable dt = DbHelper.ExecuteQuery(query);

            if (dt.Rows.Count > 0)
            {
                DataRow row = dt.Rows[0];
                return new User(
                    row["username"].ToString(),
                    row["password"].ToString(),
                    row["userrole"].ToString(),
                    double.Parse(row["balance"].ToString())
                );
            }
            return null;
        }

        public static void AddUser(User user)
        {
            string query = $"INSERT INTO users (username, password, userrole, balance) " +
                           $"VALUES ('{user.GetUsername()}', '{user.GetPassword()}',
'{user.GetUserRole()}', {user.GetBalance()})";
            DbHelper.ExecuteNonQuery(query);
            usersList = null;
        }

        public static bool DeleteAccount(string username)
        {
            string query1 = $"DELETE FROM users WHERE username = '{username}'";
            string query2 = $"DELETE FROM debit_cards WHERE username = '{username}'";
            string query3 = $"DELETE FROM admin_requests WHERE username = '{username}'";
            string query4 = $"DELETE FROM loan WHERE username = '{username}'";
            DbHelper.ExecuteNonQuery(query2);
            DbHelper.ExecuteNonQuery(query3);
            DbHelper.ExecuteNonQuery(query4);
            int rowsAffected = DbHelper.ExecuteNonQuery(query1);
            if (rowsAffected > 0) usersList = null;
            return rowsAffected > 0;
        }

        public static User GetUser(string username)
        {
            string query = $"SELECT * FROM users WHERE username = '{username}'";
            DataTable dt = DbHelper.ExecuteQuery(query);

            if (dt.Rows.Count > 0)
```

```
        {
            DataRow row = dt.Rows[0];
            return new User(
                row["username"].ToString(),
                row["password"].ToString(),
                row["userrole"].ToString(),
                double.Parse(row["balance"].ToString())
            );
        }
        return null;
    }

    public static List<User> GetAllUsers()
    {
        List<User> users = new List<User>();
        string query = "SELECT * FROM users";
        DataTable dt = DbHelper.ExecuteQuery(query);

        foreach (DataRow row in dt.Rows)
        {
            users.Add(new User(
                row["username"].ToString(),
                row["password"].ToString(),
                row["userrole"].ToString(),
                double.Parse(row["balance"].ToString())
            ));
        }
        return users;
    }


    public static void UpdateBalance(string username, double newBalance)
    {
        string query = $"UPDATE users SET balance = {newBalance} WHERE username =
'{username}'";
        DbHelper.ExecuteNonQuery(query);
    }

    public static void UpdatePassword(string username, ststringewPassword)
    {
        string query = $"UPDATE users SET password = '{newPassword}' WHERE username =
'{username}'";
        DbHelper.ExecuteNonQuery(query);
        usersList = null;
    }

    public static void UpdateUserRole(string username, ststringewRole)
    {
```

```
            string query = $"UPDATE users SET userrole = '{newRole}' WHERE username =
'{username}'";
            DbHelper.ExecuteNonQuery(query);
            usersList = null;
        }

        public static DataTable GetAllUsersDataTable()
        {
            return DbHelper.ExecuteQuery("SELECT * FROM users");
        }

        public static DataTable SearchUsersDataTable(string searchTerm)
        {
            return DbHelper.ExecuteQuery($"SELECT * FROM users WHERE username LIKE
'%{searchTerm}%'");
        }

        public static DataTable GetTop5RecentUsersDataTable()
        {
            return DbHelper.ExecuteQuery("SELECT TOP 5 * FROM users ORDER BY username
DESC");
        }
    }
}
```

DL / TransactionDL.cs

```
using BankUI.BL;
using System;
using System.Collections.Generic;
using System.Data;

namespace BankUI.DL
{
    public class TransactionDl
    {
        private static List<Transaction> transactionList = null;

        public static List<Transaction> Transactions
        {
            get
            {
                if (transactionList == null) transactionList = GetAllTransactions();
                return transactionList;
            }
        }

        public static void AddTransaction(Transaction trans)
        {
```

```
        string query = $"INSERT INTO transactions (sender, receiver, amount, transaction_date) " +
                    $"VALUES ('{trans.GetSender().GetUsername()}',
'{trans.GetReceiver().GetUsername()}', {trans.GetAmount()}, '{trans.GetTransactionDate():yyyy-
MM-dd HH:mm:ss}')";
        DbHelper.ExecuteNonQuery(query);
        transactionList = null;
    }

    public static List<Transaction> GetAllTransactions()
    {
        List<Transaction> list = new List<Transaction>();
        string query = "SELECT * FROM transactions";
        DataTable dt = DbHelper.ExecuteQuery(query);

        foreach (DataRow row in dt.Rows)
        {
            User sender = new User(row["sender"].ToString(), "", """"0);
            User receiver = new User(row["receiver"].ToString(), "", """""";
            double amount = dodoublearse(row["amount"].ToString());
            DateTime transactionDate = DateDateTimese(row["transaction_date"].ToString());
            list.Add(new Transaction(sender, receiver, amount, transactionDate));
        }
        return list;
    }

    public static List<Transaction> GetUserTransactions(string username)
    {
        List<Transaction> list = new List<Transaction>();
        string query = $"SELECT * FROM transactions WHERE sender = '{username}' OR receiver
= '{username}'";
        DataTable dt = DbHelper.ExecuteQuery(query);

        foreach (DataRow row in dt.Rows)
        {
            User sender = new User(row["sender"].ToString(), "", """"0);
            User receiver = new User(row["receiver"].ToString(), "", """""";
            double amount = dodoublearse(row["amount"].ToString());
            DateTime transactionDate = DateDateTimese(row["transaction_date"].ToString());
            list.Add(new Transaction(sender, receiver, amount, transactionDate));
        }
        return list;
    }
    public static DataTable GetTop5UserTransactions()
    {

        return DbHelper.ExecuteQuery($"SELECT Top 5* FROM transactions ORDER BY
transaction_date DESC");
```

```
        }

        public static DataTable GetTop5UserTransactionsDataTable(string username)
        {
            return DbHelper.ExecuteQuery($"SELECT TOP 5 * FROM transactions WHERE sender =
'{username}' OR receiver = '{username}' ORDER BY transaction_date DESC");
        }

        public static DataTable GetTransactionsByDateRangeDataTable(string username, DateTime
from, DateDateTime
        {
            return DbHelper.ExecuteQuery($"SELECT * FROM transactions WHERE (sender =
'{username}' OR receiver = '{username}') AND transaction_date BETWEEN '{from:yyyy-MM-dd}'
AND '{to:yyyy-MM-dd}'");
        }

        public static DataTable GetAllTransactionsDataTable()
        {
            return DbHelper.ExecuteQuery("SELECT * FROM transactions");
        }

        public static DataTable GetTransactionsByFilterDataTable(DateTime from, DateDateTime
string userFilter)
        {
            string sql = $"SELECT * FROM transactions WHERE transaction_date BETWEEN
'{from:yyyy-MM-dd}' AND '{to:yyyy-MM-dd}'";
            if (!string.IsNullOrEmpty(userFilter)) sql += $" AND (sender LIKE '%{userFilter}%' OR
receiver LIKE '%{userFilter}%')";
            return DbHelper.ExecuteQuery(sql);
        }

    }
}
```

DL / LoanDL.cs

```
using BankUI.BL;
using System.Collections.Generic;
using System.Data;

namespace BankUI.DL
{
    public class LoanDl
    {
        private static List<Loan> loansList = null;

        public static List<Loan> LoansList
        {
            get
```

```
        {
            if (loansList == null) loansList = GetAllLoans();
            return loansList;
        }
    }

    public static void AddLoan(Loan loan)
    {
        string query = $"INSERT INTO loan (username, amount, status) " +
                       $"VALUES ('{loan.GetUser().GetUsername()}', {loan.GetAmount()},
'{loan.GetStatus()}')";
        DbHelper.ExecuteNonQuery(query);
        loansList = null;
    }

    public static List<Loan> GetAllLoans()
    {
        List<Loan> loans = new List<Loan>();
        string query = "SELECT * FROM loan";
        DataTable dt = DbHelper.ExecuteQuery(query);

        foreach (DataRow row in dt.Rows)
        {
            int id =intt.Parse(row["loanid"].ToString());
            User user = new User(row["username"].ToString(), "", """""""";
            double amount = dodoublearse(row["amount"].ToString());
            string status = row["status"].ToString();
            loans.Add(new Loan(id, user, amount, status));
        }
        return loans;
    }

    public static List<Loan> GetLoansByStatus(string status)
    {
        List<Loan> loans = new List<Loan>();
        string query = $"SELECT * FROM loan WHERE status = '{status}'";
        DataTable dt = DbHelper.ExecuteQuery(query);

        foreach (DataRow row in dt.Rows)
        {
            int id =intt.Parse(row["loanid"].ToString());
            User user = new User(row["username"].ToString(), "", """""""";
            double amount = dodoublearse(row["amount"].ToString());
            string rowStatus = row["status"].ToString();
            loans.Add(new Loan(id, user, amount, rowStatus));
        }
        return loans;
    }
```

```
        public static void UpdateLoanStatus(string newStatus, int loanId)
        {
            string query = $"UPDATE loan SET status = '{newStatus}' WHERE loanid = {loanId}";
            DbHelper.ExecuteNonQuery(query);

            if (newStatus == "Approved")
            {
                foreach (Loan loan in GetAllLoans())
                {
                    if (loan.GetId() == loanId)
                    {
                        User targetUser = UserDl.GetUser(loan.GetUser().GetUsername());
                        if (targetUser != null)
                        {
                            targetUser.Deposit(loan.GetAmount());
                            UserDl.UpdateBalance(targetUser.GetUsername(), targetUser.GetBalance());
                        }
                        break;
                    }
                }
            }

            loansList = null;
        }

        public static DataTable GetAllLoansDataTable()
        {
            return DbHelper.ExecuteQuery("SELECT * FROM loan");
        }
    }
}
```

DL / DebitCardDL.cs

```
using BankUI.BL;
using System;
using System.Collections.Generic;
using System.Data;

namespace BankUI.DL
{
    public class DebitCardDl
    {
        private static List<DebitCard> debitCardsList = null;

        public static List<DebitCard> DebitCards
        {
            get
            {
```

```
            if (debitCardsList == null) debitCardsList = GetAllCards();
            return debitCardsList;
        }
    }

    public static void AddDebitCard(DebitCard card)
    {
        string formattedDate = card.GetExpiryDate().ToString("yyyy-MM-dd");
        string query = $"INSERT INTO debit_cards (card_number, username, cvc, expiry_date) " +
                       $"VALUES ('{card.GetCardNumber()}', '{card.GetUsername()}', '{card.GetCvc()}',
'{formattedDate}')";
        DbHelper.ExecuteNonQuery(query);
        debitCardsList = null;
    }

    public static DebitCard GetUserCard(string username)
    {
        string query = $"SELECT * FROM debit_cards WHERE username = '{username}'";
        DataTable dt = DbHelper.ExecuteQuery(query);

        if (dt.Rows.Count > 0)
        {
            DataRow row = dt.Rows[0];
            return new DebitCard(
                row["card_number"].ToString(),
                row["username"].ToString(),
                row["cvc"].ToString(),
                Convert.ToDateTime(row["expiry_date"])
            );
        }
        return null;
    }

    public static List<DebitCard> GetAllCards()
    {
        List<DebitCard> list = new List<DebitCard>();
        string query = "SELECT * FROM debit_cards";
        DataTable dt = DbHelper.ExecuteQuery(query);

        foreach (DataRow row in dt.Rows)
        {
            list.Add(new DebitCard(
                row["card_number"].ToString(),
                row["username"].ToString(),
                row["cvc"].ToString(),
                Convert.ToDateTime(row["expiry_date"])
            ));
        }
```

```
            return list;
        }

        public static DataTable GetAllDebitCardsDataTable()
        {
            return DbHelper.ExecuteQuery("SELECT * FROM debit_cards");
        }
    }
}
```

DL / AdminRequestDL.cs

```
using BankUI.BL;
using System.Collections.Generic;
using System.Data;

namespace BankUI.DL
{
    public class AdminRequestDl
    {
        private static List<AdminRequest> adminRequestsList = null;

        public static List<AdminRequest> AdminRequestsList
        {
            get
            {
                if (adminRequestsList == null) adminRequestsList = GetAllRequests();
                return adminRequestsList;
            }
        }

        public static void AddRequest(AdminRequest request)
        {
            string query = $"INSERT INTO admin_requests (username, status) VALUES
('{request.GetUsername()}', '{request.GetStatus()}')";
            DbHelper.ExecuteNonQuery(query);
            adminRequestsList = null;
        }

        public static List<AdminRequest> GetAllRequests()
        {
            List<AdminRequest> list = new List<AdminRequest>();
            string query = "SELECT * FROM admin_requests";
            DataTable dt = DbHelper.ExecuteQuery(query);

            foreach (DataRow row in dt.Rows)
            {
                list.Add(new AdminRequest(
                    int.Parse(row["request_id"].ToString()),
```

```
                row["username"].ToString(),
                row["status"].ToString()
            ));
        }
        return list;
    }

    public static List<AdminRequest> GetRequestsByStatus(string status)
    {
        List<AdminRequest> filtered = new List<AdminRequest>();
        foreach (AdminRequest req in AdminRequestsList)
        {
            if (req.GetStatus() == status)
                filtered.Add(req);
        }
        return filtered;
    }

    public static AdminRequest GetPendingRequest(string username)
    {
        foreach (AdminRequest req in AdminRequestsList)
        {
            if (req.GetUsername() == username && req.GetStatus() == "Pending")
                return req;
        }
        return null;
    }

    public static void UpdateRequestStatus(int requestId, string newStatus)
    {
        string query = $"UPDATE admin_requests SET status = '{newStatus}' WHERE request_id =
{requestId}";
        DbHelper.ExecuteNonQuery(query);
        adminRequestsList = null;
    }

    public static void ProcessAdminRequest(int requestId, string username, stringewStatus)
    {
        UpdateRequestStatus(requestId, newStatus);

        if (newStatus == "Approved")
        {
            UserDl.UpdateUserRole(username, "Admin");
            UserDl.RefreshUsers();
        }
    }

    public static DataTable GetAllRequestsDataTable()
```

```
        {
            return DbHelper.ExecuteQuery("SELECT * FROM admin_requests");
        }
    }
}
```

UI / LoginForm.cs

```
using System;
using System.Windows.Forms;
using BankUI.BL;
using BankUI.DL;

namespace BankUI
{
    public partial class LoginForm : Form
    {
        public string LoggedInUser { get; set; }
        public LoginForm()
        {
            InitializeComponent();
        }

        private void btnSignIn_Click(object sender, EventArgs e)
        {
            User credentials = new User(txtUsername.Text, txtPassword.Text, "", 0);
            User user = UserDl.SignIn(credentials);
            if (user != null)
            {
                LoggedInUser = user.GetUsername();
                if (user.GetUserRole() == "Admin")
                {
                    AdminDashBoardForm adminDash = new AdminDashBoardForm(LoggedInUser);
                    adminDash.Show();
                }
                else
                {
                    CustomerDashBoardForm customerDash = new
CustomerDashBoardForm(LoggedInUser);
                    customerDash.Show();
                }
                this.Hide();
            }
            else
            {
                lblError.Text = "Invalid username or password";
                MessageBox.Show("Invalid username or password");
            }
        }
```

```
        private void llbSignUp_Click(object sender, EventArgs e)
        {
            SignUpForm signUp = new SignUpForm();
            signUp.Show();
            this.Hide();
        }
    }
}
```

UI / SignUpForm.cs

```
using System;
using System.Windows.Forms;
using BankUI.BL;
using BankUI.DL;
namespace BankUI
{
    public partial class SignUpForm : Form
    {
        public string LoggedInUser { get; set; }
        public SignUpForm() { InitializeComponent(); }
        private void btnSignUp_Click(object sender, EventArgs e)
        {
            if (txtPassword.Text != txtConfirmPassword.Text)
            {
                lblError.Text = "Passwords do not match";
                MessageBox.Show("Passwords do not match");
                return;
            }
            User user = new User(txtUsername.Text, txtPassword.Text, "Customer", 0);
            UserDl.AddUser(user);
            MessageBox.Show("Registration successful!");
            LoginForm login = new LoginForm();
            login.Show();
            this.Hide();
        }
        private void llbLogin_Click(object sender, EventArgs e)
        {
            LoginForm login = new LoginForm();
            login.Show();
            this.Hide();
        }
    }
}
```

UI / AdminDashBoardForm.cs

```
using System;
using System.Windows.Forms;
using BankUI.BL;
```

```
using BankUI.DL;
namespace BankUI
{
   public partial class AdminDashBoardForm : Form
   {
      public string LoggedInUser { get; set; }
      public AdminDashBoardForm(string user)
      {
         InitializeComponent();
         LoggedInUser = user;
         LoadStats();
         dgvRecentTx.DataSource = TransactionDl.GetTop5UserTransactions();
         dgvRecentUsers.DataSource = UserDl.GetTop5RecentUsersDataTable();
      }

      private void LoadStats()
      {
         lblcardUsersV.Text = UserDl.GetAllUsers().Count.ToString();
         lblcardTxV.Text = TransactionDl.GetAllTransactions().Count.ToString();
         lblcardLoansV.Text = LoanDl.GetLoansByStatus("Pending").Count.ToString();
         lblcardCardsV.Text = DebitCardDl.GetAllCards().Count.ToString();
      }
      private void btnNavDashboard_Click(object sender, EventArgs e) { }
      private void btnNavUsers_Click(object sender, EventArgs e)
      {
         AdminUsersForm form = new AdminUsersForm(LoggedInUser);
         form.Show();
         this.Hide();
      }
      private void btnNavTransactions_Click(object sender, EventArgs e)
      {
         AdminTransactionsForm form = new AdminTransactionsForm(LoggedInUser);
         form.Show();
         this.Hide();
      }
      private void btnNavLoans_Click(object sender, EventArgs e)
      {
         AdminLoanRequestsForm form = new AdminLoanRequestsForm(LoggedInUser);
         form.Show();
         this.Hide();
      }
      private void btnNavAdminReq_Click(object sender, EventArgs e)
      {
         AdminRequestsForm form = new AdminRequestsForm(LoggedInUser);
         form.Show();
         this.Hide();
      }
      private void btnNavDebitCard_Click(object sender, EventArgs e)
```

```
        {
            AdminDebitCardForm form = new AdminDebitCardForm(LoggedInUser);
            form.Show();
            this.Hide();
        }
        private void btnNavBankStmt_Click(object sender, EventArgs e)
        {
            AdminBankStatementForm form = new AdminBankStatementForm(LoggedInUser);
            form.Show();
            this.Hide();
        }
        private void btnNavLogout_Click(object sender, EventArgs e)
        {
            LoginForm login = new LoginForm();
            login.Show();
            this.Hide();
        }

        private void lblcardUsersV_Click(object sender, EventArgs e)
        {
            int totalusers = UserDl.GetAllUsers().Count;



        }

        private void lblcardCardsV_Click(object sender, EventArgs e)
        {

        }
    }
}
```

UI / AdminUsersForm.cs

```
using System;
using System.Windows.Forms;
using BankUI.BL;
using BankUI.DL;
namespace BankUI
{
    public partial class AdminUsersForm : Form
    {
        public string LoggedInUser { get; set; }
        public AdminUsersForm(string user)
        {
            InitializeComponent();
            LoggedInUser = user;
            dgvUsers.SelectionMode = DataGridViewSelectionMode.FullRowSelect;
            LoadUsers();
        }
```

```
private void LoadUsers() { dgvUsers.DataSource = UserDl.GetAllUsersDataTable(); }
private void btnNavDashboard_Click(object sender, EventArgs e)
{
   AdminDashBoardForm f = new AdminDashBoardForm(LoggedInUser);
   f.Show();
   this.Hide();
}

private void btnNavUsers_Click(object sender, EventArgs e) { }

private void btnNavTransactions_Click(object sender, EventArgs e)
{
   AdminTransactionsForm f = new AdminTransactionsForm(LoggedInUser);
   f.Show();
   this.Hide();
}

private void btnNavLoans_Click(object sender, EventArgs e)
{
   AdminLoanRequestsForm f = new AdminLoanRequestsForm(LoggedInUser);
   f.Show();
   this.Hide();
}

private void btnNavAdminReq_Click(object sender, EventArgs e)
{
   AdminRequestsForm f = new AdminRequestsForm(LoggedInUser);
   f.Show();
   this.Hide();
}

private void btnNavDebitCard_Click(object sender, EventArgs e)
{
   AdminDebitCardForm f = new AdminDebitCardForm(LoggedInUser);
   f.Show();
   this.Hide();
}

private void btnNavBankStmt_Click(object sender, EventArgs e)
{
   AdminBankStatementForm f = new AdminBankStatementForm(LoggedInUser);
   f.Show();
   this.Hide();
}

private void btnNavLogout_Click(object sender, EventArgs e)
{
   LoginForm login = new LoginForm();
```

```
            login.Show();
            this.Hide();
        }
        private void btnSearch_Click(object sender, EventArgs e)
        {
            if (string.IsNullOrWhiteSpace(txtSearch.Text))
            {
                LoadUsers();
            }
            else
            {
                dgvUsers.DataSource = UserDl.SearchUsersDataTable(txtSearch.Text);
            }
        }
        private void btnDelete_Click(object sender, EventArgs e)
        {
            if (dgvUsers.SelectedRows.Count > 0)
            {
                string username = dgvUsers.SelectedRows[0].Cells["username"].Value.ToString();
                UserDl.DeleteAccount(username);
                MessageBox.Show("User deleted successfully");
                LoadUsers();
            }
        }
        private void btnUpdatePwd_Click(object sender, EventArgs e)
        {
            if (dgvUsers.SelectedRows.Count > 0)
            {
                string username = dgvUsers.SelectedRows[0].Cells["username"].Value.ToString();
                string newPwd = PromptForPassword("Update Password", $"Enter new pa"Enter new
password for {username}:"
                if (!string.IsNullOrEmpty(newPwd))
                {
                    UserDl.UpdatePassword(username, newPwd);
                    MessageBox.Show("Password updated successfully");
                    LoadUsers();
                }
            }
            else
            {
                MessageBox.Show("Please select a user first");
            }
        }

        private string PromptForPassword(ststringitle, stristringmptText)
        {
            Form form = new Form();
            Label label = new Label();
```

```
            TextBox textBox = new TextBox();
            Button buttonOk = new Button();
            Button buttonCancel = new Button();

            form.Text = title;
            label.Text = promptText;
            buttonOk.Text = "OK";
            buttonCancel.Text = "Cancel";
            buttonOk.DialogResult = DialogResult.OK;
            buttonCancel.DialogResult = DialogResult.Cancel;

            label.SetBounds(9, 20, 372, 13);
            textBox.SetBounds(12, 36, 372, 20);
            buttonOk.SetBounds(228, 72, 75, 23);
            buttonCancel.SetBounds(309, 72, 75, 23);

            label.AutoSize = true;
            form.ClientSize = new System.Drawing.Size(396, 107);
            form.Controls.AddRange(new Control[] { label, textBox, buttonOk, buttonCancel });
            form.FormBorderStyle = FormBorderStyle.FixedDialog;
            form.StartPosition = FormStartPosition.CenterScreen;
            form.MinimizeBox = false;
            form.MaximizeBox = false;
            form.AcceptButton = buttonOk;
            form.CancelButton = buttonCancel;

            DialogResult dialogResult = form.ShowDialog();
            return dialogResult == DialogResult.OK ? textBox.Text : "";
        }
    }
}
```

UI / AdminTransactionsForm.cs

```
using System;
using System.Windows.Forms;
using BankUI.BL;
using BankUI.DL;
namespace BankUI
{
    public partial class AdminTransactionsForm : Form
    {
        public string LoggedInUser { get; set; }
        public AdminTransactionsForm(string user)
        {
            InitializeComponent();
            LoggedInUser = user;
            dgvTransactions.DataSource = TransactionDl.GetAllTransactionsDataTable();
        }
```

```
private void btnNavDashboard_Click(object sender, EventArgs e)
{
    AdminDashBoardForm f = new AdminDashBoardForm(LoggedInUser);
    f.Show();
    this.Hide();
}

private void btnNavUsers_Click(object sender, EventArgs e)
{
    AdminUsersForm f = new AdminUsersForm(LoggedInUser);
    f.Show();
    this.Hide();
}

private void btnNavTransactions_Click(object sender, EventArgs e) { }

private void btnNavLoans_Click(object sender, EventArgs e)
{
    AdminLoanRequestsForm f = new AdminLoanRequestsForm(LoggedInUser);
    f.Show();
    this.Hide();
}

private void btnNavAdminReq_Click(object sender, EventArgs e)
{
    AdminRequestsForm f = new AdminRequestsForm(LoggedInUser);
    f.Show();
    this.Hide();
}

private void btnNavDebitCard_Click(object sender, EventArgs e)
{
    AdminDebitCardForm f = new AdminDebitCardForm(LoggedInUser);
    f.Show();
    this.Hide();
}

private void btnNavBankStmt_Click(object sender, EventArgs e)
{
    AdminBankStatementForm f = new AdminBankStatementForm(LoggedInUser);
    f.Show();
    this.Hide();
}

private void btnNavLogout_Click(object sender, EventArgs e)
{
    LoginForm login = new LoginForm();
    login.Show();
```

```
            this.Hide();
        }
        private void btnFilter_Click(object sender, EventArgs e)
        {
            dgvTransactions.DataSource =
TransactionDl.GetTransactionsByFilterDataTable(dtpFrom.Value, dtpTo.Value, txtUserFilter.Text);
        }
    }
}
```

UI / AdminLoanRequestsForm.cs

```
using System;
using System.Windows.Forms;
using BankUI.BL;
using BankUI.DL;
namespace BankUI
{
    public partial class AdminLoanRequestsForm : Form
    {
        public string LoggedInUser { get; set; }
        public AdminLoanRequestsForm(string user)
        {
            InitializeComponent();
            LoggedInUser = user;
            dgvLoans.SelectionMode = DataGridViewSelectionMode.FullRowSelect;
            LoadLoans();
        }
        private void LoadLoans()
        {
            dgvLoans.DataSource = LoanDl.GetAllLoansDataTable();
        }
        private void btnNavDashboard_Click(object sender, EventArgs e)
        {
            AdminDashBoardForm f = new AdminDashBoardForm(LoggedInUser);
            f.Show();
            this.Hide();
        }

        private void btnNavUsers_Click(object sender, EventArgs e)
        {
            AdminUsersForm f = new AdminUsersForm(LoggedInUser);
            f.Show();
            this.Hide();
        }

        private void btnNavTransactions_Click(object sender, EventArgs e)
        {
            AdminTransactionsForm f = new AdminTransactionsForm(LoggedInUser);
```

```
    f.Show();
    this.Hide();
}

private void btnNavLoans_Click(object sender, EventArgs e) { }

private void btnNavAdminReq_Click(object sender, EventArgs e)
{
    AdminRequestsForm f = new AdminRequestsForm(LoggedInUser);
    f.Show();
    this.Hide();
}

private void btnNavDebitCard_Click(object sender, EventArgs e)
{
    AdminDebitCardForm f = new AdminDebitCardForm(LoggedInUser);
    f.Show();
    this.Hide();
}

private void btnNavBankStmt_Click(object sender, EventArgs e)
{
    AdminBankStatementForm f = new AdminBankStatementForm(LoggedInUser);
    f.Show();
    this.Hide();
}

private void btnNavLogout_Click(object sender, EventArgs e)
{
    LoginForm login = new LoginForm();
    login.Show();
    this.Hide();
}
private void btnApprove_Click(object sender, EventArgs e)
{
    string status = dgvLoans.SelectedRows[0].Cells["status"].Value.ToString();
    if(status == "Approved")
    {
        MessageBox.Show("Loan is already approved");
        return;
    }
    if (dgvLoans.SelectedRows.Count > 0)
    {
        int loanId =intt.Parse(dgvLoans.SelectedRows[0].Cells["loanid"].Value.ToString());
        LoanDl.UpdateLoanStatus("Approved", loanId);
        MessageBox.Show("Loan Approved");
        LoadLoans();
    }
```

```
        }
        private void btnReject_Click(object sender, EventArgs e)
        {
            string status = dgvLoans.SelectedRows[0].Cells["status"].Value.ToString();
            if (status == "Approved")
            {
                MessageBox.Show("Loan is already approved Cant Reject Now");
                return;
            }
            if (dgvLoans.SelectedRows.Count > 0)
            {
                int loanId =intt.Parse(dgvLoans.SelectedRows[0].Cells["loanid"].Value.ToString());
                LoanDl.UpdateLoanStatus("Rejected", loanId);
                MessageBox.Show("Loan Rejected");
                LoadLoans();
            }
        }

        private void dgvLoans_CellContentClick(object sender, DataGridViewCellEventArgs e)
        {

        }
    }
}
```

UI / AdminRequestsForm.cs

```
using System;
using System.Windows.Forms;
using BankUI.BL;
using BankUI.DL;
namespace BankUI
{
    public partial class AdminRequestsForm : Form
    {
        public string LoggedInUser { get; set; }
        public AdminRequestsForm(string user)
        {
            InitializeComponent();
            LoggedInUser = user;
            dgvRequests.SelectionMode = DataGridViewSelectionMode.FullRowSelect;
            LoadRequests();
        }
        private void LoadRequests() { dgvRequests.DataSource =
AdminRequestDl.GetAllRequestsDataTable(); }
        private void btnNavDashboard_Click(object sender, EventArgs e)
        {
            AdminDashBoardForm f = new AdminDashBoardForm(LoggedInUser);
            f.Show();
```

```
        this.Hide();
    }

    private void btnNavUsers_Click(object sender, EventArgs e)
    {
        AdminUsersForm f = new AdminUsersForm(LoggedInUser);
        f.Show();
        this.Hide();
    }

    private void btnNavTransactions_Click(object sender, EventArgs e)
    {
        AdminTransactionsForm f = new AdminTransactionsForm(LoggedInUser);
        f.Show();
        this.Hide();
    }

    private void btnNavLoans_Click(object sender, EventArgs e)
    {
        AdminLoanRequestsForm f = new AdminLoanRequestsForm(LoggedInUser);
        f.Show();
        this.Hide();
    }

    private void btnNavAdminReq_Click(object sender, EventArgs e) { }

    private void btnNavDebitCard_Click(object sender, EventArgs e)
    {
        AdminDebitCardForm f = new AdminDebitCardForm(LoggedInUser);
        f.Show();
        this.Hide();
    }

    private void btnNavBankStmt_Click(object sender, EventArgs e)
    {
        AdminBankStatementForm f = new AdminBankStatementForm(LoggedInUser);
        f.Show();
        this.Hide();
    }

    private void btnNavLogout_Click(object sender, EventArgs e)
    {
        LoginForm login = new LoginForm();
        login.Show();
        this.Hide();
    }
    private void btnApprove_Click(object sender, EventArgs e)
    {
```

```
        if (dgvRequests.SelectedRows.Count > 0)
        {
            int reqId =intt.Parse(dgvRequests.SelectedRows[0].Cells["request_id"].Value.ToString());
            string username = dgvRequests.SelectedRows[0].Cells["username"].Value.ToString();
            AdminRequestDl.ProcessAdminRequest(reqId, username, "Approved");
            MessageBox.Show("Request Approved");
            LoadRequests();
        }
    }
    private void btnReject_Click(object sender, EventArgs e)
    {
        if (dgvRequests.SelectedRows.Count > 0)
        {
            int reqId =intt.Parse(dgvRequests.SelectedRows[0].Cells["request_id"].Value.ToString());
            string username = dgvRequests.SelectedRows[0].Cells["username"].Value.ToString();
            AdminRequestDl.ProcessAdminRequest(reqId, username, "Rejected");
            MessageBox.Show("Request Rejected");
            LoadRequests();
        }
    }
  }
}
```

UI / AdminDebitCardForm.cs

```
using System;
using System.Windows.Forms;
using BankUI.BL;
using BankUI.DL;
namespace BankUI
{
  public partial class AdminDebitCardForm : Form
  {
    public string LoggedInUser { get; set; }
    public AdminDebitCardForm(string user)
    {
      InitializeComponent();
      LoggedInUser = user;
      dgvDebitCards.DataSource = DebitCardDl.GetAllDebitCardsDataTable();
    }
    private void btnNavDashboard_Click(object sender, EventArgs e)
    {
      AdminDashBoardForm f = new AdminDashBoardForm(LoggedInUser);
      f.Show();
      this.Hide();
    }

    private void btnNavUsers_Click(object sender, EventArgs e)
    {
```

```
        AdminUsersForm f = new AdminUsersForm(LoggedInUser);
        f.Show();
        this.Hide();
    }

    private void btnNavTransactions_Click(object sender, EventArgs e)
    {
        AdminTransactionsForm f = new AdminTransactionsForm(LoggedInUser);
        f.Show();
        this.Hide();
    }

    private void btnNavLoans_Click(object sender, EventArgs e)
    {
        AdminLoanRequestsForm f = new AdminLoanRequestsForm(LoggedInUser);
        f.Show();
        this.Hide();
    }

    private void btnNavAdminReq_Click(object sender, EventArgs e)
    {
        AdminRequestsForm f = new AdminRequestsForm(LoggedInUser);
        f.Show();
        this.Hide();
    }

    private void btnNavDebitCard_Click(object sender, EventArgs e) { }

    private void btnNavBankStmt_Click(object sender, EventArgs e)
    {
        AdminBankStatementForm f = new AdminBankStatementForm(LoggedInUser);
        f.Show();
        this.Hide();
    }

    private void btnNavLogout_Click(object sender, EventArgs e)
    {
        LoginForm login = new LoginForm();
        login.Show();
        this.Hide();
    }

    private void dgvDebitCards_CellContentClick(object sender, DataGridViewCellEventArgs e)
    {

    }
  }
}
```

UI / AdminBankStatementForm.cs

```
using System;
using System.Windows.Forms;
using BankUI.BL;
using BankUI.DL;
namespace BankUI
{
   public partial class AdminBankStatementForm : Form
   {
      public string LoggedInUser { get; set; }
      public AdminBankStatementForm(string user)
      {
         InitializeComponent();
         LoggedInUser = user;
      }
      private void btnNavDashboard_Click(object sender, EventArgs e)
      {
         AdminDashBoardForm f = new AdminDashBoardForm(LoggedInUser);
         f.Show();
         this.Hide();
      }

      private void btnNavUsers_Click(object sender, EventArgs e)
      {
         AdminUsersForm f = new AdminUsersForm(LoggedInUser);
         f.Show();
         this.Hide();
      }

      private void btnNavTransactions_Click(object sender, EventArgs e)
      {
         AdminTransactionsForm f = new AdminTransactionsForm(LoggedInUser);
         f.Show();
         this.Hide();
      }

      private void btnNavLoans_Click(object sender, EventArgs e)
      {
         AdminLoanRequestsForm f = new AdminLoanRequestsForm(LoggedInUser);
         f.Show();
         this.Hide();
      }

      private void btnNavAdminReq_Click(object sender, EventArgs e)
      {
         AdminRequestsForm f = new AdminRequestsForm(LoggedInUser);
         f.Show();
```

```
            this.Hide();
        }

        private void btnNavDebitCard_Click(object sender, EventArgs e)
        {
            AdminDebitCardForm f = new AdminDebitCardForm(LoggedInUser);
            f.Show();
            this.Hide();
        }

        private void btnNavBankStmt_Click(object sender, EventArgs e) { }

        private void btnNavLogout_Click(object sender, EventArgs e)
        {
            LoginForm login = new LoginForm();
            login.Show();
            this.Hide();
        }
        private void btnGenerate_Click(object sender, EventArgs e)
        {
            dgvStatement.DataSource =
TransactionDl.GetTransactionsByDateRangeDataTable(txtUsername.Text, dtpFrom.Value,
dtpTo.Value);
        }
        private void btnExport_Click(object sender, EventArgs e) { MessageBox.Show("Statement
Generated"); }
    }
}
```

UI / CustomerDashBoardForm.cs

```
using System;
using System.Windows.Forms;
using BankUI.BL;
using BankUI.DL;
namespace BankUI
{
    public partial class CustomerDashBoardForm : Form
    {
        public string LoggedInUser { get; set; }
        public CustomerDashBoardForm(string user)
        {
            InitializeComponent();
            LoggedInUser = user;
            User u = UserDl.GetUser(LoggedInUser);
            lblBalance.Text = $"Balance: ${u.GetBalance()}";
            dgvRecentTx.DataSource =
TransactionDl.GetTop5UserTransactionsDataTable(LoggedInUser);
        }
        private void btnNavDashboard_Click(object sender, EventArgs e) { }
```

```
private void btnNavTransfer_Click(object sender, EventArgs e)
{
    CustomerTransferForm f = new CustomerTransferForm(LoggedInUser);
    f.Show();
    this.Hide();
}

private void btnNavDebitCard_Click(object sender, EventArgs e)
{
    CustomerApplyDebitCardForm f = new CustomerApplyDebitCardForm(LoggedInUser);
    f.Show();
    this.Hide();
}

private void btnNavLoan_Click(object sender, EventArgs e)
{
    CustomerLoanRequestForm f = new CustomerLoanRequestForm(LoggedInUser);
    f.Show();
    this.Hide();
}

private void btnNavAdminReq_Click(object sender, EventArgs e)
{
    CustomerApplyAdminForm f = new CustomerApplyAdminForm(LoggedInUser);
    f.Show();
    this.Hide();
}

private void btnNavBalance_Click(object sender, EventArgs e)
{
    CustomerCheckBalanceForm f = new CustomerCheckBalanceForm(LoggedInUser);
    f.Show();
    this.Hide();
}

private void btnNavBankStmt_Click(object sender, EventArgs e)
{
    CustomerBankStatementForm f = new CustomerBankStatementForm(LoggedInUser);
    f.Show();
    this.Hide();
}

private void btnNavLogout_Click(object sender, EventArgs e)
{
    LoginForm login = new LoginForm();
    login.Show();
    this.Hide();
```

```
        }
        private void btnQuickTransfer_Click(object sender, EventArgs e) {
btnNavTransfer_Click(sender, e); }
        private void btnQuickLoan_Click(object sender, EventArgs e) { btnNavLoan_Click(sender, e);
}
        private void btnQuickCard_Click(object sender, EventArgs e) {
btnNavDebitCard_Click(sender, e); }
        private void btnQuickStmt_Click(object sender, EventArgs e) {
btnNavBankStmt_Click(sender, e); }

        private void lblBalance_Click(object sender, EventArgs e)
        {

        }
    }
}
```

UI / CustomerTransferForm.cs

```
using System;
using System.Windows.Forms;
using BankUI.BL;
using BankUI.DL;
namespace BankUI
{
    public partial class CustomerTransferForm : Form
    {
        public string LoggedInUser { get; set; }
        public CustomerTransferForm(string user)
        {
            InitializeComponent();
            LoggedInUser = user;
        }
        private void btnNavDashboard_Click(object sender, EventArgs e)
        {
            CustomerDashBoardForm f = new CustomerDashBoardForm(LoggedInUser);
            f.Show();
            this.Hide();
        }

        private void btnNavTransfer_Click(object sender, EventArgs e) { }

        private void btnNavDebitCard_Click(object sender, EventArgs e)
        {
            CustomerApplyDebitCardForm f = new CustomerApplyDebitCardForm(LoggedInUser);
            f.Show();
            this.Hide();
        }
```

```
private void btnNavLoan_Click(object sender, EventArgs e)
{
   CustomerLoanRequestForm f = new CustomerLoanRequestForm(LoggedInUser);
   f.Show();
   this.Hide();
}

private void btnNavAdminReq_Click(object sender, EventArgs e)
{
   CustomerApplyAdminForm f = new CustomerApplyAdminForm(LoggedInUser);
   f.Show();
   this.Hide();
}

private void btnNavBalance_Click(object sender, EventArgs e)
{
   CustomerCheckBalanceForm f = new CustomerCheckBalanceForm(LoggedInUser);
   f.Show();
   this.Hide();
}

private void btnNavBankStmt_Click(object sender, EventArgs e)
{
   CustomerBankStatementForm f = new CustomerBankStatementForm(LoggedInUser);
   f.Show();
   this.Hide();
}

private void btnNavLogout_Click(object sender, EventArgs e)
{
   LoginForm login = new LoginForm();
   login.Show();
   this.Hide();
}
private void btnTransfer_Click(object sender, EventArgs e)
{
   if (double.TryParse(txtAmount.Text, out dodoublemount))
   {
      User senderUser = UserDl.GetUser(LoggedInUser);
      if (senderUser.GetBalance() >= amount)
      {
         User receiverUser = UserDl.GetUser(txtReceiver.Text);
         if (receiverUser != null)
         {
            senderUser.Withdraw(amount);
            receiverUser.Deposit(amount);
```

```
                UserDl.UpdateBalance(senderUser.GetUsername(), senderUser.GetBalance());
                UserDl.UpdateBalance(receiverUser.GetUsername(), receiverUser.GetBalance());

                Transaction tx = new Transaction(senderUser, receiverUser, amount,
DateTime.Now);
                TransactionDl.AddTransaction(tx);
                lblStatus.Text = "Transfer Successful";
                MessageBox.Show("Transfer Successful");
            }
            else { MessageBox.Show("Receiver not found"); }
        }
        else { MessageBox.Show("Insufficient balance"); }
    }
    else { MessageBox.Show("Please enter a valid amount"); }
  }
}
```

UI / CustomerCheckBalanceForm.cs

```
using System;
using System.Windows.Forms;
using BankUI.BL;
using BankUI.DL;
namespace BankUI
{
  public partial class CustomerCheckBalanceForm : Form
  {
    public string LoggedInUser { get; set; }
    public CustomerCheckBalanceForm(string user)
    {
      InitializeComponent();
      LoggedInUser = user;
      UpdateBalanceDisplay();
    }

    private void UpdateBalanceDisplay()
    {
      User u = UserDl.GetUser(LoggedInUser);
      lblBalance.Text = $"Rs. {u.GetBalance():N2}";
    }

    private void btnNavDashboard_Click(object sender, EventArgs e)
    {
      CustomerDashBoardForm f = new CustomerDashBoardForm(LoggedInUser);
      f.Show();
      this.Hide();
    }
    private void btnNavTransfer_Click(object sender, EventArgs e)
```

```
        {
            CustomerTransferForm f = new CustomerTransferForm(LoggedInUser);
            f.Show();
            this.Hide();
        }
        private void btnNavDebitCard_Click(object sender, EventArgs e) {
CustomerApplyDebitCardForm f = new CustomerApplyDebitCardForm(LoggedInUser); f.Show();
this.Hide(); }
        private void btnNavLoan_Click(object sender, EventArgs e) { CustomerLoanRequestForm f =
new CustomerLoanRequestForm(LoggedInUser); f.Show(); this.Hide(); }
        private void btnNavAdminReq_Click(object sender, EventArgs e) {
CustomerApplyAdminForm f = new CustomerApplyAdminForm(LoggedInUser); f.Show();
this.Hide(); }
        private void btnNavBalance_Click(object sender, EventArgs e) { }
        private void btnNavBankStmt_Click(object sender, EventArgs e) {
CustomerBankStatementForm f = new CustomerBankStatementForm(LoggedInUser); f.Show();
this.Hide(); }
        private void btnNavLogout_Click(object sender, EventArgs e) { LoginForm login = new
LoginForm(); login.Show(); this.Hide(); }

        private void lblBalance_Click(object sender, EventArgs e) { }

        private void btnTransfer_Click(object sender, EventArgs e)
        {
            if (double.TryParse(txtAmount.Text, out dodoublemount) && amount > 0)
            {
                User u = UserDl.GetUser(LoggedInUser);
                double newBalance = u.GetBalance() + amount;


                UserDl.UpdateBalance(LoggedInUser, newBalance);
                UpdateBalanceDisplay();
                txtAmount.Clear();
                MessageBox.Show("Successfully deposited", "Deposit Success", M"Deposit Success"OK,
MessageBoxIcon.Information);
            }
            else
            {
                MessageBox.Show("Please enter a valid positive amount.", "Invalid Input",
MessageBoxButtons."Invalid Input"con.Warning);
            }
        }
    }
}
```

UI / CustomerApplyDebitCardForm.cs

```
using System;
using System.Windows.Forms;
```

```
using BankUI.BL;
using BankUI.DL;
namespace BankUI
{
   public partial class CustomerApplyDebitCardForm : Form
   {
      public string LoggedInUser { get; set; }
      public CustomerApplyDebitCardForm(string user)
      {
         InitializeComponent();
         LoggedInUser = user;
         DebitCard card = DebitCardDl.GetUserCard(LoggedInUser);
         if (card != null)
         {
            lblCardNumber.Text = card.GetCardNumber();
            lblCardHolder.Text = card.GetUsername();
            lblCardExpiry.Text = card.GetExpiryDate().ToString("MM/yy");
            btnApply.Enabled = false;
         }
      }
      private void btnNavDashboard_Click(object sender, EventArgs e)
      {
         CustomerDashBoardForm f = new CustomerDashBoardForm(LoggedInUser);
         f.Show();
         this.Hide();
      }

      private void btnNavTransfer_Click(object sender, EventArgs e)
      {
         CustomerTransferForm f = new CustomerTransferForm(LoggedInUser);
         f.Show();
         this.Hide();
      }

      private void btnNavDebitCard_Click(object sender, EventArgs e) { }

      private void btnNavLoan_Click(object sender, EventArgs e)
      {
         CustomerLoanRequestForm f = new CustomerLoanRequestForm(LoggedInUser);
         f.Show();
         this.Hide();
      }

      private void btnNavAdminReq_Click(object sender, EventArgs e)
      {
         CustomerApplyAdminForm f = new CustomerApplyAdminForm(LoggedInUser);
         f.Show();
         this.Hide();
```

```
    }

    private void btnNavBalance_Click(object sender, EventArgs e)
    {
        CustomerCheckBalanceForm f = new CustomerCheckBalanceForm(LoggedInUser);
        f.Show();
        this.Hide();
    }

    private void btnNavBankStmt_Click(object sender, EventArgs e)
    {
        CustomerBankStatementForm f = new CustomerBankStatementForm(LoggedInUser);
        f.Show();
        this.Hide();
    }

    private void btnNavLogout_Click(object sender, EventArgs e)
    {
        LoginForm login = new LoginForm();
        login.Show();
        this.Hide();
    }
    private void btnApply_Click(object sender, EventArgs e)
    {

        User currentUser = UserDl.GetUser(LoggedInUser);
        if (DebitCard.HasSufficientBalance(currentUser))
        {
            DebitCard card = new DebitCard(currentUser);
            lblCardNumber.Text = card.GetCardNumber();
            lblCardHolder.Text = card.GetUsername();
            lblCardExpiry.Text = card.GetExpiryDate().ToString("MM/yy");
            lblStatus.Text = "Card Applied Successfully";
            btnApply.Enabled = false;
            MessageBox.Show("Debit Card Issued Successfully! 500 has been deducted from your
account.");
        }
        else
        {
            MessageBox.Show("Insufficient balance! You need at least 500 to apply for a card.");
        }
    }
  }
}
```

UI / CustomerLoanRequestForm.cs

```
using System;
using System.Windows.Forms;
```

```
using BankUI.BL;
using BankUI.DL;
namespace BankUI
{
   public partial class CustomerLoanRequestForm : Form
   {
      public string LoggedInUser { get; set; }
      public CustomerLoanRequestForm(string user)
      {
         InitializeComponent();
         LoggedInUser = user;
      }
      private void btnNavDashboard_Click(object sender, EventArgs e)
      {
         CustomerDashBoardForm f = new CustomerDashBoardForm(LoggedInUser);
         f.Show();
         this.Hide();
      }

      private void btnNavTransfer_Click(object sender, EventArgs e)
      {
         CustomerTransferForm f = new CustomerTransferForm(LoggedInUser);
         f.Show();
         this.Hide();
      }

      private void btnNavDebitCard_Click(object sender, EventArgs e)
      {
         CustomerApplyDebitCardForm f = new CustomerApplyDebitCardForm(LoggedInUser);
         f.Show();
         this.Hide();
      }

      private void btnNavLoan_Click(object sender, EventArgs e) { }

      private void btnNavAdminReq_Click(object sender, EventArgs e)
      {
         CustomerApplyAdminForm f = new CustomerApplyAdminForm(LoggedInUser);
         f.Show();
         this.Hide();
      }

      private void btnNavBalance_Click(object sender, EventArgs e)
      {
         CustomerCheckBalanceForm f = new CustomerCheckBalanceForm(LoggedInUser);
         f.Show();
         this.Hide();
      }
```

```
        private void btnNavBankStmt_Click(object sender, EventArgs e)
        {
            CustomerBankStatementForm f = new CustomerBankStatementForm(LoggedInUser);
            f.Show();
            this.Hide();
        }

        private void btnNavLogout_Click(object sender, EventArgs e)
        {
            LoginForm login = new LoginForm();
            login.Show();
            this.Hide();
        }
        private void btnApply_Click(object sender, EventArgs e)
        {

            if (!double.TryParse(txtAmount.Text, out dodoublemount))
            {

                MessageBox.Show("Please enter a valid numeric amount.", "Invalid Input",
MessageBoxButtons"Invalid Input"Icon.Warning);
                return;
            }
            if (amount <= 0)
            {
                MessageBox.Show("The loan amount must be greater than zero.", "Invalid Amount",
MessageBoxButtons.OK, "Invalid Amount"arning);
                return;
            }
            Loan loan = new Loan(0, UserDl.GetUser(LoggedInUser), amount, "Pending");
            LoanDl.AddLoan(loan);
            lblStatus.Text = "Loan Requested";
            MessageBox.Show("Loan Request Submitted Successfully");
        }

        private void txtAmount_TextChanged(object sender, EventArgs e)
        {

        }
    }
}
```

UI / CustomerApplyAdminForm.cs

```
using System;
using System.Windows.Forms;
using BankUI.BL;
```

```
using BankUI.DL;
namespace BankUI
{
   public partial class CustomerApplyAdminForm : Form
   {
      public string LoggedInUser { get; set; }
      public CustomerApplyAdminForm(string user)
      {
         InitializeComponent();
         LoggedInUser = user;
      }
      private void btnNavDashboard_Click(object sender, EventArgs e)
      {
         CustomerDashBoardForm f = new CustomerDashBoardForm(LoggedInUser);
         f.Show();
         this.Hide();
      }

      private void btnNavTransfer_Click(object sender, EventArgs e)
      {
         CustomerTransferForm f = new CustomerTransferForm(LoggedInUser);
         f.Show();
         this.Hide();
      }

      private void btnNavDebitCard_Click(object sender, EventArgs e)
      {
         CustomerApplyDebitCardForm f = new CustomerApplyDebitCardForm(LoggedInUser);
         f.Show();
         this.Hide();
      }

      private void btnNavLoan_Click(object sender, EventArgs e)
      {
         CustomerLoanRequestForm f = new CustomerLoanRequestForm(LoggedInUser);
         f.Show();
         this.Hide();
      }

      private void btnNavAdminReq_Click(object sender, EventArgs e) { }

      private void btnNavBalance_Click(object sender, EventArgs e)
      {
         CustomerCheckBalanceForm f = new CustomerCheckBalanceForm(LoggedInUser);
         f.Show();
         this.Hide();
      }
```

```
        private void btnNavBankStmt_Click(object sender, EventArgs e)
        {
            CustomerBankStatementForm f = new CustomerBankStatementForm(LoggedInUser);
            f.Show();
            this.Hide();
        }

        private void btnNavLogout_Click(object sender, EventArgs e)
        {
            LoginForm login = new LoginForm();
            login.Show();
            this.Hide();
        }
        private void btnApply_Click(object sender, EventArgs e)
        {
            AdminRequest req = new AdminRequest(0, LoggedInUser, "Pending");
            AdminRequestDl.AddRequest(req);
            lblStatus.Text = "Request Submitted";
            MessageBox.Show("Admin Request Submitted Successfully");
        }
    }
}
```

UI / CustomerBankStatementForm.cs

```
using System;
using System.Windows.Forms;
using BankUI.BL;
using BankUI.DL;
namespace BankUI
{
    public partial class CustomerBankStatementForm : Form
    {
        public string LoggedInUser { get; set; }
        public CustomerBankStatementForm(string user)
        {
            InitializeComponent();
            LoggedInUser = user;
        }
        private void btnNavDashboard_Click(object sender, EventArgs e)
        {
            CustomerDashBoardForm f = new CustomerDashBoardForm(LoggedInUser);
            f.Show();
            this.Hide();
        }

        private void btnNavTransfer_Click(object sender, EventArgs e)
        {
            CustomerTransferForm f = new CustomerTransferForm(LoggedInUser);
```

```
        f.Show();
        this.Hide();
    }

    private void btnNavDebitCard_Click(object sender, EventArgs e)
    {
        CustomerApplyDebitCardForm f = new CustomerApplyDebitCardForm(LoggedInUser);
        f.Show();
        this.Hide();
    }

    private void btnNavLoan_Click(object sender, EventArgs e)
    {
        CustomerLoanRequestForm f = new CustomerLoanRequestForm(LoggedInUser);
        f.Show();
        this.Hide();
    }

    private void btnNavAdminReq_Click(object sender, EventArgs e)
    {
        CustomerApplyAdminForm f = new CustomerApplyAdminForm(LoggedInUser);
        f.Show();
        this.Hide();
    }

    private void btnNavBalance_Click(object sender, EventArgs e)
    {
        CustomerCheckBalanceForm f = new CustomerCheckBalanceForm(LoggedInUser);
        f.Show();
        this.Hide();
    }

    private void btnNavBankStmt_Click(object sender, EventArgs e) { }

    private void btnNavLogout_Click(object sender, EventArgs e)
    {
        LoginForm login = new LoginForm();
        login.Show();
        this.Hide();
    }
    private void btnView_Click(object sender, EventArgs e)
    {
        dgvStatement.DataSource =
TransactionDl.GetTransactionsByDateRangeDataTable(LoggedInUser, dtpFrom.Value,
dtpTo.Value);
    }

    private void dtpTo_ValueChanged(object sender, EventArgs e)
```

```
        {

        }
    }
}
```

Weaknesses in the Windows Forms Application

1. Passwords are stored in plain text inside the users table. Anyone with direct database access can read every user's credentials. The application should hash passwords using a standard algorithm such as SHA-256 before storing them.

2. All SQL queries are assembled using string concatenation with user input inserted directly. This exposes the entire database to SQL injection attacks. Parameterized queries using SqlParameter objects should replace all inline string-built queries throughout the DL layer.

3. The application has no real-time refresh mechanism. If two users are logged in simultaneously, changes made by one, such as a balance update or a loan approval, are not visible to the other until they navigate away and return to that form.

4. There is no session timeout. If a user leaves their account open without logging out, anyone who accesses the same machine can use the account freely. An inactivity timer that triggers automatic logout would address this.

5. The Export CSV button in AdminBankStatementForm shows a placeholder MessageBox instead of actually writing a file. The export feature is visually present but not functionally complete.

6. The CustomerApplyDebitCardForm creates the DebitCard object directly inside the form using new DebitCard(user) instead of using the static DebitCard.IssueCard() method defined in the BL class. This bypasses the balance and duplicate checks in the BL and is inconsistent with the three-layer architecture used everywhere else.

Future Directions

1. Implement password hashing by passing passwords through SHA-256 or BCrypt before saving to the database and comparing hashed values during login, so that plaintext credentials are never stored.

2. Replace all string-concatenated SQL queries with parameterized queries throughout every DL class, which will eliminate the SQL injection vulnerability entirely.

3. Complete the Export CSV feature in AdminBankStatementForm using StreamWriter to write the DataGridView contents to a .csv file selected through a SaveFileDialog.

4. Add an inactivity timer in each dashboard form that triggers automatic logout after a configurable period, such as ten minutes without input.

5. Refactor CustomerApplyDebitCardForm to call DebitCard.IssueCard() instead of constructing the DebitCard object directly, so that all business rules are enforced through the BL layer consistently.

6. Add an interest calculation feature that runs on loan approval and schedules monthly deductions from the customer's balance, making the loan system financially realistic.

7. Add a confirmation dialog before all destructive actions such as deleting a user account or rejecting a loan, to prevent accidental data loss.